

CHƯƠNG 1: NETWORKING

1.1. Giới thiệu

Mạng máy tính (Computer Network) là hệ thống cho phép các thiết bị trao đổi dữ liệu với nhau thông qua các giao thức truyền thông. Trong phát triển Backend, kiến thức về mạng giúp lập trình viên hiểu được cách trình duyệt giao tiếp với máy chủ, cách dữ liệu được truyền qua Internet và cách các dịch vụ hoạt động trong môi trường thực tế.

Đối với lập trình viên Backend, các kiến thức quan trọng cần nắm bao gồm TCP/IP, HTTP, HTTPS, DNS, Domain, Cookie, Session, JWT, Load Balancer và quá trình xử lý một yêu cầu từ trình duyệt đến máy chủ.

OSI MODEL:

Layer	Chức năng
Application	HTTP, HTTPS
Transport	TCP, UDP
Network	IP
Data Link	MAC Address
Physical	Dây mạng

1.2. Giao thức TCP/IP

TCP/IP là bộ giao thức nền tảng của Internet, chịu trách nhiệm truyền tải dữ liệu giữa các thiết bị.

Trong đó:

- IP (Internet Protocol) chịu trách nhiệm định tuyến dữ liệu giữa các thiết bị trên mạng.
- TCP (Transmission Control Protocol) chịu trách nhiệm đảm bảo dữ liệu được truyền đầy đủ, đúng thứ tự và không bị mất.

TCP được sử dụng trong hầu hết các ứng dụng yêu cầu tính chính xác cao như HTTP, HTTPS, SSH và cơ sở dữ liệu.

Đặc điểm của TCP

- Thiết lập kết nối trước khi truyền dữ liệu.

- Đảm bảo dữ liệu đến đúng đích.
- Đảm bảo đúng thứ tự các gói tin.
- Tự động gửi lại gói tin nếu bị mất.

Ứng dụng

- HTTP/HTTPS
- SSH
- FTP
- Kết nối cơ sở dữ liệu

1.3. TCP Three-way Handshake

Trước khi truyền dữ liệu, TCP phải thiết lập kết nối giữa Client và Server thông qua cơ chế Three-way Handshake.

Quá trình này gồm ba bước:

1. Client gửi gói tin SYN để yêu cầu thiết lập kết nối.
2. Server phản hồi bằng gói SYN-ACK để xác nhận.
3. Client gửi gói ACK để hoàn tất quá trình kết nối.

Sau khi hoàn thành ba bước trên, hai bên mới bắt đầu truyền dữ liệu.

Ghi nhớ: HTTP luôn phải thiết lập kết nối TCP trước khi gửi Request.

1.4. UDP

UDP (User Datagram Protocol) là giao thức truyền dữ liệu không yêu cầu thiết lập kết nối và không đảm bảo dữ liệu được truyền đầy đủ.

Do không cần cơ chế xác nhận và gửi lại gói tin nên UDP có tốc độ truyền nhanh hơn TCP nhưng độ tin cậy thấp hơn.

Đặc điểm

- Không thiết lập kết nối.
- Không kiểm tra mất gói tin.
- Không đảm bảo thứ tự dữ liệu.
- Độ trễ thấp.

Ứng dụng

- Video Call
- Livestream

- Game Online
- DNS

1.5. So sánh TCP và UDP

TCP	UDP
Có thiết lập kết nối	Không thiết lập kết nối
Đảm bảo dữ liệu	Không đảm bảo dữ liệu
Có cơ chế ACK	Không có ACK
Tốc độ chậm hơn	Tốc độ nhanh hơn
HTTP, HTTPS	Streaming, Game, DNS

1.6. HTTP

HTTP (HyperText Transfer Protocol) là giao thức được sử dụng để trao đổi dữ liệu giữa Client và Server.

Mỗi lần người dùng truy cập một website hoặc gọi API đều thực hiện thông qua HTTP Request và HTTP Response.

Một HTTP Request thường bao gồm:

- Method
- URL
- Header
- Body (nếu có)

Server sẽ xử lý yêu cầu và trả về HTTP Response bao gồm:

- Status Code
- Header
- Body

1.6.1. HTTP Methods

HTTP cung cấp nhiều phương thức để thực hiện các thao tác khác nhau đối với tài nguyên.

Method	Chức năng
GET	Lấy dữ liệu
POST	Tạo mới dữ liệu

PUT	Cập nhật toàn bộ dữ liệu
PATCH	Cập nhật một phần dữ liệu
DELETE	Xóa dữ liệu

1.6.2. HTTP Status Code

Status Code là mã trạng thái mà Server trả về sau khi xử lý Request.

Các nhóm trạng thái thường gặp:

Nhóm	Ý nghĩa
2xx	Thành công
3xx	Chuyển hướng
4xx	Lỗi từ Client
5xx	Lỗi từ Server

Một số mã phổ biến:

Status	Ý nghĩa
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
500	Internal Server Error

1.7. HTTPS

HTTPS (HyperText Transfer Protocol Secure) là phiên bản bảo mật của HTTP.

HTTPS sử dụng giao thức TLS/SSL để mã hóa dữ liệu trước khi truyền trên Internet.

Việc mã hóa giúp:

- Bảo vệ dữ liệu khỏi bị nghe lén.
- Ngăn chặn việc chỉnh sửa dữ liệu trong quá trình truyền.
- Xác thực danh tính của máy chủ thông qua chứng chỉ số (SSL Certificate).

Ngày nay hầu hết các website đều sử dụng HTTPS thay cho HTTP.

1.8. DNS

DNS (Domain Name System) là hệ thống chuyển đổi tên miền thành địa chỉ IP.

Ví dụ:

google.com

↓

142.250.xxx.xxx

Nhờ DNS, người dùng chỉ cần nhớ tên miền thay vì ghi nhớ địa chỉ IP của máy chủ.

1.9. Domain

Domain (Tên miền) là địa chỉ định danh của một website trên Internet.

Ví dụ:

api.example.com

Trong đó:

- com là miền cấp cao (Top Level Domain).
- example là tên miền.
- api là tên miền phụ (Subdomain).

1.10. Điều gì xảy ra khi người dùng truy cập một website?

Khi người dùng nhập địa chỉ website trên trình duyệt và nhấn Enter, hệ thống sẽ thực hiện các bước sau:

1. Trình duyệt kiểm tra bộ nhớ đệm (Cache).
2. Nếu chưa có địa chỉ IP, trình duyệt gửi yêu cầu đến DNS để phân giải tên miền.
3. Thiết lập kết nối TCP với máy chủ.
4. Nếu sử dụng HTTPS, thực hiện TLS Handshake.
5. Gửi HTTP Request đến máy chủ.
6. Máy chủ xử lý yêu cầu và truy vấn cơ sở dữ liệu nếu cần.
7. Máy chủ trả về HTTP Response.
8. Trình duyệt tải HTML, CSS, JavaScript và hiển thị giao diện cho người dùng.

Quá trình trên diễn ra trong thời gian rất ngắn, thường chỉ vài trăm mili giây.

Ví dụ:

Browser



DNS



TCP Handshake



TLS Handshake



HTTP Request



Load Balancer



Backend



Database



HTTP Response



Browser Render

CHƯƠNG 2: BACKEND ARCHITECTURE

2.1. API Design

2.1.1. API (Application Programming Interface)

API (Application Programming Interface) là tập hợp các quy tắc và giao thức cho phép các ứng dụng giao tiếp và trao đổi dữ liệu với nhau. Trong phát triển Backend, API đóng vai trò là cầu nối giữa Client (Web, Mobile) và Server.

Khi người dùng thực hiện một thao tác trên giao diện, Client sẽ gửi một yêu cầu (Request) đến API. Server tiếp nhận yêu cầu, xử lý nghiệp vụ, truy vấn cơ sở dữ liệu nếu cần và trả về kết quả thông qua Response.

2.1.2. RESTful API

RESTful API là phong cách thiết kế API dựa trên kiến trúc REST (Representational State Transfer). Mỗi tài nguyên (Resource) được biểu diễn bằng một URL và các thao tác trên tài nguyên được thực hiện thông qua các HTTP Method.

RESTful API giúp hệ thống dễ mở rộng, dễ bảo trì và dễ tích hợp giữa nhiều nền tảng khác nhau.

Một RESTful API thường tuân theo các nguyên tắc:

- Sử dụng HTTP Method đúng mục đích.
- Mỗi URL đại diện cho một Resource.
- Không lưu trạng thái của Client trên Server (Stateless).
- Trả về dữ liệu theo định dạng thống nhất (thường là JSON).

Ví dụ:

HTTP Method	URL	Chức năng
GET	/users	Lấy danh sách người dùng
GET	/users/1	Lấy thông tin người dùng
POST	/users	Tạo người dùng mới
PATCH	/users/1	Cập nhật thông tin
DELETE	/users/1	Xóa người dùng

2.1.3. Resource Naming

Resource Naming là quy tắc đặt tên URL trong RESTful API.

Một số nguyên tắc thường sử dụng:

- Sử dụng danh từ thay vì động từ.
- Sử dụng số nhiều cho Resource.
- Không đưa hành động vào URL.
- Sử dụng dấu gạch ngang (-) nếu tên gồm nhiều từ.

Ví dụ:

Đúng:

- GET /users
- GET /products
- POST /orders

Không nên:

- GET /getUsers
- POST /createUser
- DELETE /deleteProduct

Việc đặt tên thống nhất giúp API dễ đọc và dễ sử dụng.

2.1.4. API Versioning

API Versioning là kỹ thuật quản lý nhiều phiên bản API nhằm đảm bảo các ứng dụng cũ vẫn hoạt động khi API được cập nhật.

Các cách versioning phổ biến:

URI Versioning

- /api/v1/users
- /api/v2/users

Header Versioning

- Accept: application/vnd.company.v2+json

Trong thực tế, URI Versioning được sử dụng phổ biến nhất vì đơn giản và dễ quản lý.

2.1.5. Idempotent Method

Idempotent là tính chất mà khi cùng một yêu cầu được gửi nhiều lần thì kết quả cuối cùng của hệ thống vẫn không thay đổi.

Ví dụ:

DELETE /users/10

Nếu gửi yêu cầu nhiều lần:

- Lần đầu: người dùng bị xóa.
- Các lần sau: người dùng đã không còn tồn tại.

Kết quả cuối cùng vẫn giống nhau.

Một số HTTP Method có tính Idempotent:

- GET
- PUT
- DELETE

POST mặc định không có tính Idempotent, vì mỗi lần gửi có thể tạo thêm dữ liệu mới.

Ví dụ:

POST /orders

Gửi hai lần có thể tạo hai đơn hàng khác nhau.

2.1.6. Stateless

Stateless là đặc điểm quan trọng của REST.

Mỗi Request phải chứa đầy đủ thông tin để Server xử lý mà không phụ thuộc vào Request trước đó.

Ví dụ:

- GET /profile
- Authorization: Bearer <JWT>

Server xác thực trực tiếp thông qua JWT trong Request mà không cần ghi nhớ trạng thái của Client.

Ưu điểm:

- Dễ mở rộng hệ thống.
- Dễ triển khai nhiều Server.
- Giảm việc lưu Session trên Server.

2.2. Backend Architecture

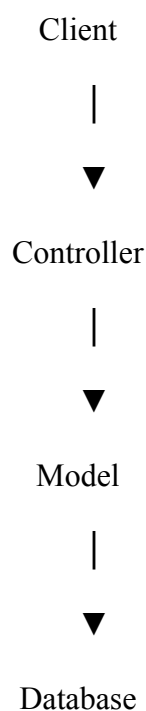
Kiến trúc Backend là cách tổ chức các thành phần trong hệ thống nhằm giúp mã nguồn rõ ràng, dễ bảo trì và dễ mở rộng.

2.2.1. MVC (Model - View - Controller)

MVC là mô hình kiến trúc chia ứng dụng thành ba thành phần chính.

- Model: quản lý dữ liệu và nghiệp vụ liên quan đến dữ liệu.
- View: hiển thị giao diện cho người dùng.
- Controller: tiếp nhận Request và điều phối xử lý.

Luồng hoạt động:



Trong Backend API, View thường được thay bằng dữ liệu JSON trả về Client.

Ưu điểm:

- Phân tách trách nhiệm.
- Dễ bảo trì.
- Dễ mở rộng.

2.2.2. Thin Controller - Fat Service

Đây là kiến trúc phổ biến trong NestJS.

Controller chỉ thực hiện các nhiệm vụ:

- Nhận Request.

- Validate dữ liệu.
- Gọi Service.
- Trả Response.

Service chịu trách nhiệm xử lý toàn bộ nghiệp vụ.

Ví dụ:



Controller:

```

@Post()

create(@Body() dto: CreateBookingDto) {

    return this.bookingService.create(dto);

}
  
```

Service:

```

async create(dto: CreateBookingDto) {

    // Validate nghiệp vụ

    // Transaction

    // Gọi Repository

}
  
```

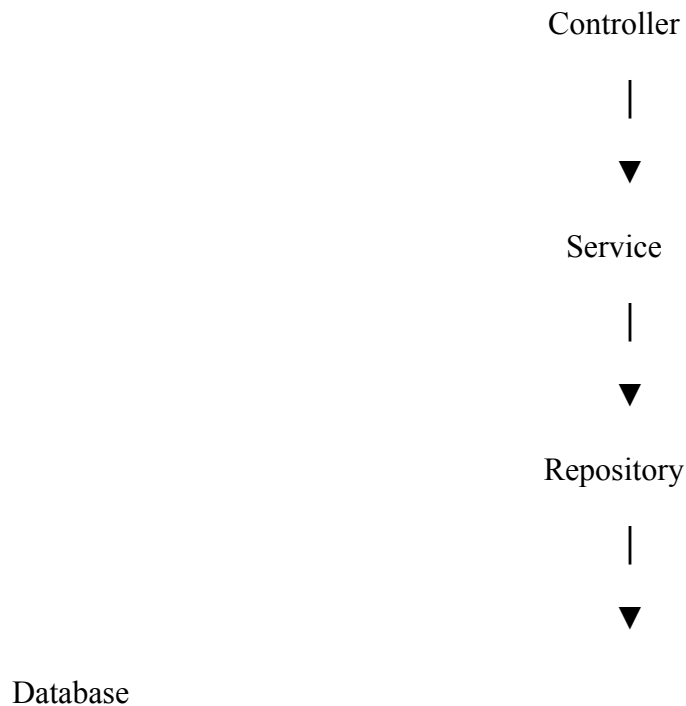
Ưu điểm:

- Controller ngắn gọn.
- Nghiệp vụ tập trung tại Service.
- Dễ kiểm thử.

2.2.3. Repository Pattern

Repository Pattern là mẫu thiết kế giúp tách riêng việc truy cập cơ sở dữ liệu khỏi phần xử lý nghiệp vụ.

Luồng xử lý:



Service không làm việc trực tiếp với Database mà thông qua Repository.

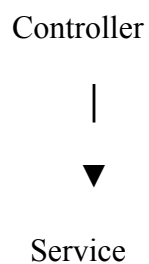
Ưu điểm:

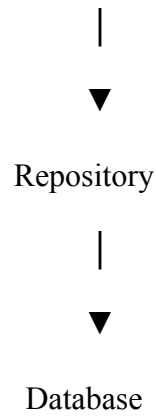
- Giảm phụ thuộc vào Database.
- Dễ thay đổi ORM.
- Dễ viết Unit Test.

2.2.4. Layered Architecture

Layered Architecture chia ứng dụng thành nhiều tầng, mỗi tầng đảm nhận một nhiệm vụ riêng.

Thông thường gồm:





Mỗi tầng chỉ giao tiếp với tầng liền kề.

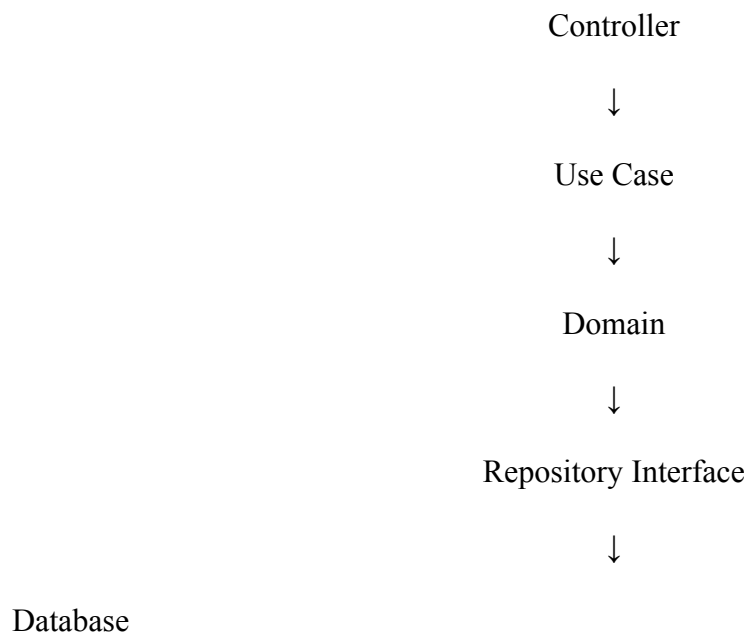
Ưu điểm:

- Dễ quản lý.
- Dễ mở rộng.
- Dễ bảo trì.

2.2.5. Clean Architecture

Clean Architecture là kiến trúc giúp giảm sự phụ thuộc giữa nghiệp vụ và công nghệ.

Các tầng thường gặp:



Trong kiến trúc này:

- Nghiệp vụ không phụ thuộc Framework.
- Có thể thay đổi Database hoặc Framework mà ít ảnh hưởng đến Business Logic.

Đối với vị trí Intern hoặc Junior, cần hiểu mục tiêu chính của Clean Architecture là tách biệt Business Logic khỏi các thành phần bên ngoài để tăng khả năng bảo trì và kiểm thử.

2.3. Scalability

Scalability là khả năng mở rộng hệ thống khi số lượng người dùng hoặc lưu lượng truy cập tăng lên.

2.3.1. Vertical Scaling

Vertical Scaling (Scale Up) là nâng cấp tài nguyên của một máy chủ.

Ví dụ:

- Tăng RAM.
- Tăng CPU.
- Tăng ổ cứng.

Ưu điểm:

- Triển khai đơn giản.
- Không cần thay đổi kiến trúc.

Nhược điểm:

- Chi phí cao.
- Bị giới hạn bởi cấu hình phần cứng.

2.3.2. Horizontal Scaling

Horizontal Scaling (Scale Out) là tăng số lượng máy chủ.

Ví dụ:

- Server 1
- Server 2
- Server 3

Các Server cùng xử lý Request.

Ưu điểm:

- Dễ mở rộng.
- Tăng khả năng chịu tải.
- Tăng tính sẵn sàng.

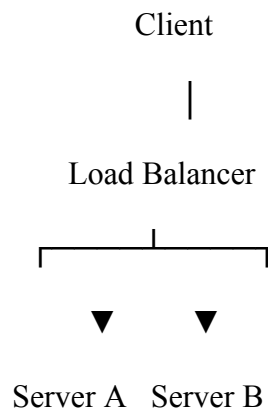
Nhược điểm:

- Kiến trúc phức tạp hơn.
- Cần Load Balancer.

2.3.3. Load Balancer

Load Balancer là thành phần chịu trách nhiệm phân phối Request đến nhiều máy chủ khác nhau nhằm cân bằng tải.

Ví dụ:



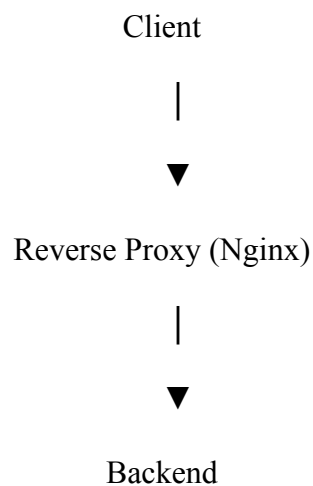
Lợi ích:

- Phân phối đều lưu lượng truy cập.
- Tránh quá tải một máy chủ.
- Tăng khả năng sẵn sàng khi có Server gặp sự cố.

2.3.4. Reverse Proxy

Reverse Proxy là máy chủ trung gian đứng trước Backend Server.

Luồng hoạt động:



Một số chức năng:

- Chuyển tiếp Request.
- SSL Termination.
- Nén dữ liệu.
- Cache nội dung.
- Kết hợp với Load Balancer.

2.3.5. Caching

Caching là kỹ thuật lưu tạm dữ liệu để giảm số lần truy cập đến cơ sở dữ liệu hoặc dịch vụ bên ngoài.

Ví dụ:

Client

|



Redis Cache

|

Cache Hit?

└─ Có → Trả dữ liệu

└─ Không



Database

Lợi ích:

- Giảm thời gian phản hồi.
- Giảm tải cho Database.
- Tăng hiệu năng hệ thống.

Tuy nhiên, dữ liệu trong Cache cần được đồng bộ với Database. Nếu không có cơ chế cập nhật hoặc xóa Cache phù hợp (Cache Invalidation), hệ thống có thể trả về dữ liệu cũ.

CHƯƠNG 3: DATABASE

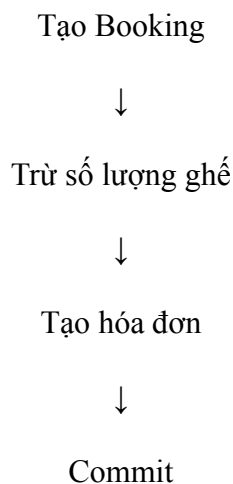
3.1. Transaction

Khái niệm: Transaction là một nhóm các thao tác trên cơ sở dữ liệu (thêm, sửa, xóa) được xử lý như một khối thống nhất: hoặc tất cả cùng thành công, hoặc tất cả cùng thất bại. Đây là công cụ quan trọng để đảm bảo dữ liệu luôn ở trạng thái đúng đắn, đặc biệt trong các nghiệp vụ có nhiều bước liên quan đến nhau, ví dụ: chuyển tiền giữa hai tài khoản (trừ tiền tài khoản A và cộng tiền tài khoản B phải cùng thành công hoặc cùng thất bại).

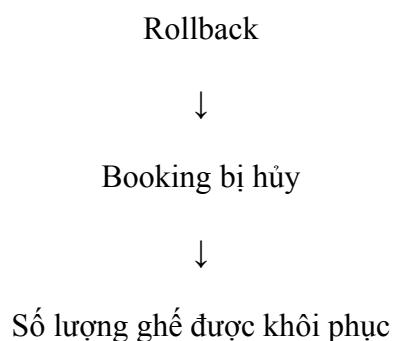
Nếu tất cả các thao tác thành công, Transaction sẽ được Commit.

Nếu một thao tác thất bại, toàn bộ các thay đổi sẽ được Rollback.

Ví dụ:



Nếu bước tạo hóa đơn xảy ra lỗi:



Nhờ Transaction, dữ liệu luôn giữ được tính nhất quán.

Khi nào cần Transaction

Transaction nên được sử dụng khi có nhiều thao tác liên quan đến cùng một nghiệp vụ.

Ví dụ:

- Đặt vé.
- Đặt phòng.
- Chuyển tiền.
- Thanh toán.
- Tạo đơn hàng.

Nếu không sử dụng Transaction, hệ thống có thể rơi vào trạng thái dữ liệu không đồng nhất.

3.1.1. ACID

ACID là bốn tính chất mà một transaction cần đảm bảo:

Tính chất	Ý nghĩa
Atomicity (Tính nguyên tử)	Các thao tác trong transaction được xem là một khối không thể chia nhỏ; nếu một bước lỗi, toàn bộ transaction bị hủy bỏ.
Consistency (Tính nhất quán)	Dữ liệu trước và sau transaction luôn tuân thủ các ràng buộc đã định nghĩa (khóa ngoại, ràng buộc unique...).
Isolation (Tính cô lập)	Các transaction chạy đồng thời không ảnh hưởng lẫn nhau, mỗi transaction "cảm giác" như đang chạy một mình.
Durability (Tính bền vững)	Khi transaction đã commit thành công, dữ liệu được lưu trữ vĩnh viễn, kể cả khi hệ thống gặp sự cố ngay sau đó.

3.1.2. Các lệnh cơ bản

BEGIN: Bắt đầu một transaction mới.

COMMIT: Xác nhận và lưu vĩnh viễn toàn bộ thay đổi trong transaction.

ROLLBACK: Hủy bỏ toàn bộ thay đổi, đưa dữ liệu về trạng thái trước khi transaction bắt đầu.

Ví dụ minh họa (NestJS + TypeORM):

```
async transferMoney(fromId: number, toId: number, amount: number) {

  const queryRunner = this.dataSource.createQueryRunner();

  await queryRunner.connect();

  await queryRunner.startTransaction(); // BEGIN
```

```

try {

    await queryRunner.manager.decrement(Account, { id: fromId }, 'balance', amount);

    await queryRunner.manager.increment(Account, { id: toId }, 'balance', amount);

    await queryRunner.commitTransaction(); // COMMIT

} catch (error) {

    await queryRunner.rollbackTransaction(); // ROLLBACK

    throw error;

} finally {

    await queryRunner.release();

}

}

```

3.2. Isolation Level

Isolation Level quy định mức độ "nhìn thấy" dữ liệu chưa hoàn tất của nhau giữa các transaction chạy song song. Mức cô lập càng cao thì dữ liệu càng an toàn nhưng hiệu năng hệ thống càng giảm, vì cơ sở dữ liệu phải khóa dữ liệu nhiều hơn để đảm bảo tính đúng đắn.

Mức độ	Mô tả	Dirty Read	Non-repeatable Read	Phantom Read
Read Uncommitted	Cho phép đọc cả dữ liệu chưa được commit của transaction khác	Có	Có	Có
Read Committed	Chỉ đọc dữ liệu đã được commit	Không	Có	Có
Repeatable Read	Đảm bảo đọc lại nhiều lần trong cùng transaction luôn cho kết quả giống nhau	Không	Không	Có (tùy hệ CSDL)
Serializable	Mức cô lập cao nhất, các transaction chạy như thể tuần tự lần lượt	Không	Không	Không

Các vấn đề thường gặp khi thiếu cô lập

- **Dirty Read (Đọc dữ liệu bẩn):** Một transaction đọc được dữ liệu mà transaction khác đang thay đổi nhưng chưa commit. Nếu transaction kia rollback, dữ liệu vừa đọc trở nên vô nghĩa.
- **Non-repeatable Read (Đọc không lặp lại được):** Trong cùng một transaction, đọc cùng một dòng dữ liệu hai lần nhưng nhận được hai kết quả khác nhau, do transaction khác đã sửa và commit ở giữa hai lần đọc.
- **Phantom Read:** Trong cùng một transaction, thực hiện lại cùng một truy vấn điều kiện (ví dụ: đếm số đơn hàng) nhưng nhận được số dòng khác nhau, do transaction khác đã thêm/xóa dòng dữ liệu thỏa điều kiện đó.
- **Lost Update (Mất cập nhật):** Hai transaction cùng đọc một giá trị, cùng tính toán rồi cùng ghi đè; kết quả của transaction ghi sau sẽ "đè mất" kết quả của transaction ghi trước, dù cả hai đều thao tác hợp lệ. Đây là vấn đề rất hay gặp trong thực tế (ví dụ: hai người cùng lúc trừ số lượng tồn kho của một sản phẩm).

3.3. Locking

Locking là cơ chế cơ sở dữ liệu sử dụng để ngăn nhiều transaction cùng thay đổi một dữ liệu tại một thời điểm, nhằm tránh các vấn đề đã nêu ở trên.

Loại khóa	Mô tả	Khi nào phù hợp
Optimistic Lock (Khóa lạc quan)	Không khóa dữ liệu ngay khi đọc; chỉ kiểm tra dữ liệu có bị thay đổi hay không tại thời điểm ghi (thường qua một trường version). Nếu phát hiện xung đột thì từ chối cập nhật.	Hệ thống có tỉ lệ xung đột thấp, ưu tiên hiệu năng đọc.
Pessimistic Lock (Khóa bi quan)	Khóa dữ liệu ngay khi bắt đầu đọc để chuẩn bị sửa, các transaction khác phải chờ cho đến khi khóa được giải phóng.	Hệ thống có tỉ lệ xung đột cao, dữ liệu quan trọng cần chắc chắn tuyệt đối.
Row Lock (Khóa theo dòng)	Chỉ khóa (các) dòng dữ liệu cụ thể đang được thao tác, thay vì khóa toàn bảng, giúp tăng khả năng xử lý song song.	Hầu hết các hệ quản trị CSDL hiện đại (MySQL InnoDB, PostgreSQL) đều dùng row lock làm mặc định.
Distributed Lock (Khóa phân tán)	Cơ chế khóa hoạt động trên nhiều tiến trình/máy chủ khác nhau (không chỉ trong phạm vi một CSDL), thường triển	Hệ thống có nhiều service/instance cùng truy cập một tài nguyên dùng chung.

	khởi bằng Redis (Redlock) khi hệ thống chạy nhiều instance backend.	
--	---	--

Ví dụ Optimistic Lock (NestJS + TypeORM, dùng trường version):

TypeScript

```
@Entity()

export class Product {

    @PrimaryGeneratedColumn()

    id: number;

    @Column()

    stock: number;

    @VersionColumn() // TypeORM tự tăng version mỗi lần update

    version: number;

}

// Khi update, TypeORM tự thêm điều kiện WHERE version = <version cũ>

// Nếu version đã thay đổi (bị người khác update trước) => ném lỗi OptimisticLockVersionMismatchError
```

Ví dụ Pessimistic Lock (SQL thuần, dùng trong transaction):

Sql

```
BEGIN;

SELECT stock FROM product WHERE id = 1 FOR UPDATE; -- khóa dòng dữ liệu

UPDATE product SET stock = stock - 1 WHERE id = 1;
```

3.4. Index

Khái niệm: Index là một cấu trúc dữ liệu bổ sung giúp cơ sở dữ liệu tìm kiếm dữ liệu nhanh hơn, tương tự như mục lục của một cuốn sách giúp người đọc không phải lật từng trang để tìm nội dung.

Loại Index	Mô tả
Clustered Index	Quyết định thứ tự vật lý lưu trữ dữ liệu trên đĩa; mỗi bảng chỉ có duy nhất một clustered index (thường là khóa chính).
Non-clustered Index	Là một cấu trúc riêng biệt, lưu trữ giá trị của cột được đánh index kèm con trỏ trỏ đến dòng dữ liệu gốc; một bảng có thể có nhiều non-clustered index.
UNIQUE Index	Đảm bảo giá trị trong cột (hoặc nhóm cột) không bị trùng lặp, đồng thời cũng giúp tăng tốc tìm kiếm như index thông thường.
Composite Index	Index được tạo trên nhiều cột cùng lúc, hữu ích khi truy vấn thường xuyên lọc/sắp xếp theo nhiều cột kết hợp.
Covering Index	Index chứa đủ tất cả các cột mà truy vấn cần, giúp cơ sở dữ liệu trả kết quả trực tiếp từ index mà không cần truy cập vào bảng gốc, giúp truy vấn nhanh hơn đáng kể.

Khi nào nên dùng Index:

- Cột thường xuyên xuất hiện trong mệnh đề WHERE, JOIN, ORDER BY.
- Bảng có số lượng dữ liệu lớn và tần suất đọc (SELECT) nhiều hơn ghi.
- Cột có độ phân tán giá trị cao (nhiều giá trị khác nhau), ví dụ email, mã đơn hàng.

Khi nào không nên lạm dụng Index:

- Bảng có tần suất ghi (INSERT/UPDATE/DELETE) rất cao, vì mỗi lần ghi dữ liệu, các index liên quan cũng phải được cập nhật theo, làm chậm thao tác ghi.
- Cột có độ phân tán giá trị thấp (ví dụ cột giới tính chỉ có 2-3 giá trị) thường không mang lại hiệu quả tốt khi đánh index.
- Bảng có kích thước nhỏ, quét toàn bộ bảng đã đủ nhanh, việc thêm index không mang lại lợi ích đáng kể mà còn tốn thêm dung lượng lưu trữ.

3.5. Scaling Database

Khi hệ thống phát triển, một cơ sở dữ liệu duy nhất có thể không còn đủ khả năng đáp ứng lượng truy cập, do đó cần các chiến lược mở rộng:

- **Read Replica (Bản sao chỉ đọc):** Tạo ra một hoặc nhiều bản sao của cơ sở dữ liệu chính (Master), các bản sao này chỉ phục vụ mục đích đọc dữ liệu (SELECT), trong khi mọi thao tác ghi (INSERT/UPDATE/DELETE) vẫn đi qua Master. Cách này giúp giảm tải cho Master khi hệ thống có lượng đọc lớn hơn nhiều so với ghi.
- **Sharding (Phân mảnh dữ liệu — khái niệm):** Là kỹ thuật chia nhỏ dữ liệu ra thành nhiều cơ sở dữ liệu độc lập (gọi là các "shard"), mỗi shard chỉ lưu một phần dữ liệu (ví dụ: chia theo khu vực địa lý, hoặc theo khoảng ID người dùng). Sharding giúp hệ thống mở rộng theo chiều ngang khi dữ liệu quá lớn để một máy chủ duy nhất lưu trữ, nhưng đánh đổi lại là độ phức tạp trong việc quản lý và truy vấn dữ liệu tăng lên đáng kể.

CHƯƠNG 4: Software Engineering

4.1. Giới thiệu

Software Engineering (Kỹ nghệ phần mềm) là lĩnh vực nghiên cứu các phương pháp, nguyên tắc và quy trình nhằm xây dựng phần mềm có chất lượng cao, dễ bảo trì và dễ mở rộng. Trong phát triển Backend, việc áp dụng các nguyên lý thiết kế giúp giảm sự phụ thuộc giữa các thành phần, tăng khả năng tái sử dụng mã nguồn và thuận tiện cho việc kiểm thử.

Chương này trình bày các nguyên lý hướng đối tượng (OOP), nguyên tắc SOLID, các quy tắc viết mã sạch (Clean Code) và một số Design Pattern thường được sử dụng trong NestJS.

4.2. OOP

4.2.1. Khái niệm

Lập trình hướng đối tượng (Object-Oriented Programming – OOP) là một mô hình lập trình dựa trên việc tổ chức phần mềm thành các đối tượng – những thực thể kết hợp giữa dữ liệu (thuộc tính) và hành vi (phương thức). Thay vì coi chương trình chỉ là một chuỗi các lệnh tuần tự, OOP cho phép mô hình hóa thế giới thực thành các lớp đối tượng có quan hệ với nhau, từ đó giúp mã nguồn trở nên trực quan, tái sử dụng cao và dễ quản lý hơn khi quy mô dự án tăng lên.

Đối Tượng Object)

Đối tượng trong OOP gồm 2 thành phần chính:

- thuộc tính (Attribute): là những thông tin đặc điểm đối tượng
- phương thức (Method): là những hành vi mà đối tượng có thể thực hiện

Dễ hình dung ta có một đối tượng là smartphone đối tượng này sẽ có:

- thuộc tính: màu sắc, bộ nhớ, hệ điều hành...
- phương thức: gọi điện, chụp ảnh, nhắn tin, ghi âm...

Lớp (Class)

lớp là sự trừu tượng hoá của đối tượng. Những đối tượng có đặc tính tương tự nhau sẽ được tập hợp thành một lớp. Lớp cũng sẽ bao gồm 2 thuộc tính và phương thức.

Một đối tượng sẽ được xem là 1 thực thể của lớp.

Tiếp nối ví dụ ở phần đối tượng (Object) phía trên ta có lớp (Class) smartphone gồm 2 thành phần:

- thuộc tính: màu sắc, bộ nhớ, hệ điều hành...

- phương thức: gọi điện, chụp ảnh, nhắn tin, ghi âm...

các đối tượng của lớp này có thể là: iphone, Samsung, Oppo, Huawei...

4.2.2. Encapsulation

Đây là tính chất được sử dụng khá nhiều. Tính kế thừa cho phép xây dựng một lớp mới (lớp Con), kế thừa và tái sử dụng các thuộc tính phương thức dựa trên lớp cũ (lớp Cha) đã có trước đó.

Các lớp Con kế thừa toàn bộ thành phần của lớp Cha và không cần phải định nghĩa lại. Lớp có thể mở rộng các thành phần kế thừa hoặc bổ sung những thành phần mới

Ví dụ:

- Lớp Cha là smartphone, có các thuộc tính: màu sắc, bộ nhớ, hệ điều hành...
- Các lớp Con là iPhone, Samsung, Oppo cũng có các thuộc tính màu sắc và bộ nhớ, hệ điều hành...

Ví dụ trong NestJS (TypeScript):

```
class BankAccount {  
  
  private balance: number = 0; // không thể truy cập trực tiếp từ bên ngoài  
  
  public deposit(amount: number): void {  
  
    if (amount > 0) {  
  
      this.balance += amount;  
  
    }  
  
  }  
  
  public getBalance(): number {  
  
    return this.balance; // truy cập có kiểm soát  
  
  }  
  
}
```

4.2.3. Inheritance

Tính kế thừa cho phép định nghĩa một lớp mới dựa trên một lớp đã có, thừa hưởng các thuộc tính và phương thức, đồng thời có thể bổ sung hoặc thay đổi hành vi. Kế thừa giúp tái sử dụng mã nguồn, xây dựng hệ thống phân cấp rõ ràng và dễ mở rộng.

```
class BaseController {  
  
  handleSuccess(res, data) {  
  
    return res.status(200).json(data);  
  
  }  
  
  handleError(res, error) {  
  
    return res.status(500).json({ message: error.message });  
  
  }  
  
}  
  
class UserController extends BaseController {  
  
  async getUser(req, res) {  
  
    try {  
  
      const user = await userService.findById(req.params.id);  
  
      this.handleSuccess(res, user);  
  
    } catch (err) {  
  
      this.handleError(res, err);  
  
    }  
  
  }  
  
}
```

4.2.4. Polymorphism

Đa hình là khả năng một thực thể (phương thức hoặc đối tượng) có thể hành xử khác nhau tùy theo ngữ cảnh. Trong OOP, đa hình thường được thể hiện qua việc ghi đè phương thức (overriding) hoặc nạp chồng (overloading – trong các ngôn ngữ hỗ trợ). Nhờ đa hình, cùng một lời gọi phương thức nhưng có thể thực hiện những hành vi khác nhau tùy thuộc vào lớp thực thể của đối tượng.

```
interface PaymentService {  
    pay(amount: number): void;  
}  
  
class MomoPayment implements PaymentService {  
    pay(amount: number) { /* xử lý thanh toán Momo */ }  
}  
  
class ZaloPayPayment implements PaymentService {  
    pay(amount: number) { /* xử lý thanh toán ZaloPay */ }  
}  
  
// Ở một nơi khác, chỉ cần inject PaymentService mà không cần biết  
// implementation cụ thể  
  
class OrderController {  
    constructor(private paymentService: PaymentService) {}  
  
    checkout() { this.paymentService.pay(100); }  
}
```

Hành vi pay() sẽ được quyết định tại thời điểm chạy dựa trên implementation được cung cấp. Điều này thể hiện đa hình thông qua interface.

4.2.5. Abstraction

Trừu tượng là khả năng ẩn đi các chi tiết triển khai phức tạp, chỉ để lộ ra những chức năng cốt lõi dưới dạng giao diện hoặc lớp trừu tượng. Mục tiêu là giảm độ phức tạp, cho phép người lập trình tập trung vào việc đối tượng “làm gì” hơn là “làm như thế nào”.

Trong thực tế backend, các lớp service thường đóng vai trò là lớp trừu tượng cho tầng nghiệp vụ. Một controller chỉ cần gọi `userService.createUser(data)` mà không cần biết dữ liệu được lưu vào cơ sở dữ liệu nào hay qua những bước kiểm tra gì. Trong NestJS, khái niệm “provider” kết hợp với DI giúp thể hiện rõ tính trừu tượng.

4.3. Design Pattern

4.3.1. Dependency Injection

4.3.2. Repository Pattern

Repository Pattern là một mẫu thiết kế thuộc tầng truy xuất dữ liệu, đóng vai trò trung gian giữa logic nghiệp vụ và nguồn dữ liệu (database, API...). Pattern này tạo ra một lớp trừu tượng để thực hiện các thao tác CRUD, giúp tầng nghiệp vụ không cần biết đến câu lệnh SQL hay ORM cụ thể. Nhờ đó, mã nguồn trở nên sạch sẽ và khi thay đổi hệ quản trị cơ sở dữ liệu, chỉ cần sửa bên trong repository.

Trong NestJS, Repository Pattern thường được triển khai cùng TypeORM:

typescript

`@Entity()`

`class User {`

`@PrimaryGeneratedColumn() id: number;`

`@Column() name: string;`

`}`

`@Injectable()`

`class UserRepository extends Repository<User> {`

`// TypeORM cung cấp các method cơ bản: find, save, delete...`

`async findActiveUsers(): Promise<User[]> {`

```

    return this.find({ where: { isActive: true } });
  }
}

```

UserService chỉ gọi `this.userRepository.findActiveUsers()` mà không cần viết SQL. Nếu chuyển từ PostgreSQL sang MongoDB, chỉ cần thay đổi cấu hình TypeORM và chỉnh sửa các truy vấn đặc thù trong repository, tầng nghiệp vụ gần như không bị ảnh hưởng.

4.3.3. Factory Pattern

Factory Pattern là mẫu thiết kế thuộc nhóm khởi tạo, cung cấp một phương thức (hoặc một lớp chuyên biệt) để tạo đối tượng mà không cần chỉ định chính xác lớp cụ thể. Điều này hữu ích khi việc tạo đối tượng phức tạp hoặc cần che giấu logic lựa chọn lớp dựa trên tham số đầu vào.

Ví dụ trong ứng dụng Express.js:

javascript

```

class LoggerFactory {

  static createLogger(type) {

    switch (type) {

      case 'file': return new FileLogger();

      case 'console': return new ConsoleLogger();

      default: throw new Error('Unknown logger type');

    }

  }

}

```

```
const logger = LoggerFactory.createLogger('file');
```

Trong NestJS, Factory thường ẩn sau các custom provider như `useFactory`:

typescript

```
@Module({
```

```

providers: [

  {

    provide: 'CONFIG',

    useFactory: () => {

      return process.env.NODE_ENV === 'production'

        ? new ProductionConfig()

        : new DevelopmentConfig();

    }

  }

]

})

```

Như vậy, logic quyết định khởi tạo lớp nào được tách biệt, có thể tái sử dụng và dễ kiểm thử.

4.3.4. Strategy Pattern (NestJS dùng khá nhiều)

Strategy Pattern thuộc nhóm hành vi, cho phép định nghĩa một tập hợp các thuật toán, đóng gói mỗi thuật toán thành một lớp riêng biệt và cho phép thay đổi thuật toán một cách linh hoạt tại runtime. Pattern này tuân thủ nguyên lý Open/Closed: mở cho việc mở rộng (thêm strategy mới) nhưng đóng với việc sửa đổi code hiện có.

NestJS ứng dụng Strategy Pattern rất rõ trong hệ thống xác thực với thư viện Passport. Mỗi chiến lược xác thực (JWT, OAuth2, Local...) là một lớp riêng, cùng tuân theo interface chung và có thể “cắm” vào hệ thống một cách dễ dàng.

Ví dụ:

typescript

```
// JwtStrategy
```

```
@Injectable()
```

```

class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {

  async validate(payload: any) {

    return { userId: payload.sub, username: payload.username };

  }

}

```



```

    }
}

// LocalStrategy

@Injectable()
class LocalStrategy extends PassportStrategy(Strategy, 'local') {

    async validate(username: string, password: string) {

        const user = await this.authService.validateUser(username, password);

        if (!user) throw new UnauthorizedException();

        return user;

    }

}

```

Controller hoặc Guard chỉ cần gọi `AuthGuard('jwt')` hay `AuthGuard('local')`, phần còn lại do strategy được chọn đảm nhiệm. Khi cần thêm phương thức xác thực mới (ví dụ Google OAuth), nhà phát triển chỉ cần tạo thêm một strategy class mới mà không phải sửa đổi logic xác thực hiện tại – đúng tinh thần của Strategy Pattern.

CHƯƠNG 5: Backend Processing Techniques

Trong các hệ thống Backend hiện đại, ngoài việc xây dựng API và kết nối cơ sở dữ liệu, lập trình viên còn phải xử lý nhiều vấn đề phát sinh trong quá trình vận hành như kiểm tra dữ liệu đầu vào, đảm bảo tính nhất quán của dữ liệu, xử lý đồng thời nhiều yêu cầu và ngăn chặn các lỗi do người dùng gửi yêu cầu lặp lại.

Các kỹ thuật này giúp hệ thống hoạt động ổn định, chính xác và có khả năng mở rộng khi số lượng người dùng tăng lên.

Chương này trình bày các kỹ thuật xử lý phổ biến trong Backend, bao gồm Validation, Transaction, Concurrency, Atomic Update, Idempotency, Queue, Cache và các cơ chế hỗ trợ khác.

5.1. Validation

Khái niệm: Validation là quá trình kiểm tra dữ liệu đầu vào trước khi hệ thống thực hiện xử lý nghiệp vụ.

Mục tiêu của Validation là:

- Ngăn chặn dữ liệu không hợp lệ.
- Đảm bảo tính chính xác của dữ liệu.
- Giảm lỗi trong quá trình xử lý.
- Tăng tính bảo mật cho hệ thống.

Thông thường Validation được chia thành hai loại:

- DTO Validation.
- Business Validation.

5.1.1. DTO Validation

DTO (Data Transfer Object) là đối tượng dùng để nhận dữ liệu từ Client.

DTO Validation kiểm tra định dạng của dữ liệu trước khi chuyển vào Service.

Ví dụ:

```
export class CreateUserDto {  
  
    @IsEmail()  
  
    email: string;  
  
  
    @MinLength(6)
```

```
password: string;  
}
```

Trong ví dụ trên:

- Email phải đúng định dạng.
- Password phải có ít nhất 6 ký tự.

Nếu dữ liệu không hợp lệ, hệ thống sẽ trả về lỗi 400 Bad Request mà không tiếp tục xử lý.

Một số kiểm tra thường gặp

- Required Field.
- Email Format.
- UUID.
- Số nguyên.
- Độ dài chuỗi.
- Giá trị tối thiểu/tối đa.
- Enum.

Ưu điểm

- Phát hiện lỗi sớm.
- Giảm dữ liệu không hợp lệ.
- Mã nguồn rõ ràng.

5.1.2. Business Validation

Business Validation là quá trình kiểm tra các quy tắc nghiệp vụ của hệ thống.

Khác với DTO Validation, Business Validation không kiểm tra định dạng dữ liệu mà kiểm tra xem dữ liệu có hợp lệ theo nghiệp vụ hay không.

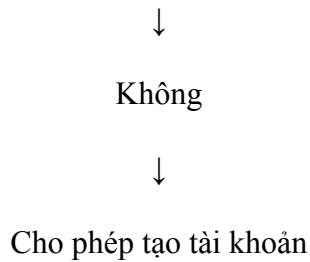
Ví dụ:

Đăng ký tài khoản

Email đúng định dạng



Email đã tồn tại?



Trong ví dụ trên:

- Email đúng định dạng là DTO Validation.
- Email đã tồn tại hay chưa là Business Validation.

Một số ví dụ khác:

- Người dùng đã đủ 18 tuổi chưa.
- Sản phẩm còn hàng không.
- Phòng còn chỗ không.
- Voucher còn hiệu lực không.
- Người dùng có quyền truy cập không.

Business Validation thường được thực hiện trong Service vì đây là nơi xử lý nghiệp vụ của hệ thống.

5.2. Transaction

Ở Chương 3, Transaction được trình bày ở góc độ lý thuyết cơ sở dữ liệu. Ở đây, ta tập trung vào cách áp dụng transaction trong code Backend: khi một nghiệp vụ cần thực hiện nhiều thao tác ghi dữ liệu liên quan đến nhau (ví dụ: tạo đơn hàng đồng thời trừ tiền kho), toàn bộ các thao tác đó nên được gói trong một transaction để đảm bảo tính toàn vẹn dữ liệu, tránh trường hợp một phần thao tác thành công còn phần khác thất bại.

typescript

```
try {  
  
  const booking = await db.transaction().execute(async (trx) => {  
  
    const updated = await decrementSlotRemaining(trx, slotId);  
  
    if (updated === 0) {  
  
      return { ok: false as const, error: BookingErrors.slotFull() };  
  
    }  
  
  }  
}
```

```
const created = await insertBooking(trx, {

  userId,

  slotId,

  idempotencyKey,

});

return { ok: true as const, booking: created };

});

if (!booking.ok) {

  return booking.error;

}

return Response.json(serializeBooking(booking.booking), { status: 201 });

} catch (error) {

  if (

    error instanceof SqliteError &&

    error.code === "SQLITE_CONSTRAINT_UNIQUE"

  ) {

    const duplicate = await findBookingByIdempotencyKey(db, idempotencyKey);

    if (duplicate) {

      return Response.json(serializeBooking(duplicate), { status: 200 });

    }

    return BookingErrors.alreadyBooked();

  }

  return BookingErrors.internal();

}
```

5.3. Concurrency

Khái niệm

Concurrency là khả năng hệ thống xử lý nhiều yêu cầu tại cùng một thời điểm.

Ví dụ:



Nếu không kiểm soát tốt, dữ liệu có thể bị sai lệch.

5.3.1. Race Condition

Race Condition xảy ra khi nhiều Request cùng truy cập và thay đổi một dữ liệu trong cùng một thời điểm, dẫn đến kết quả không mong muốn.

Ví dụ:

Ban đầu:

Stock = 1

Hai người cùng mua.

User A đọc Stock = 1

User B đọc Stock = 1

Cả hai đều tiếp tục tạo đơn hàng.

Kết quả:

Stock = -1

Đây được gọi là hiện tượng **Overselling**.

Race Condition thường xuất hiện trong:

- Booking.
- Thanh toán.

- Flash Sale.
- Chuyển tiền.

5.3.2. Lost Update

Lost Update là hiện tượng một Transaction ghi đè kết quả cập nhật của Transaction khác.

Ví dụ:

Giá trị ban đầu:

Balance = 100

Transaction A:

Đọc 100

↓

+20

↓

120

Transaction B:

Đọc 100

↓

-30

↓

70

Nếu Transaction B ghi sau cùng:

Balance = 70

Thay vì:

Balance = 90

Kết quả cập nhật của Transaction A đã bị mất.

```
// SAI - dễ gây Lost Update

const product = await productRepository.findOne(id);

product.stock = product.stock - 1;

await productRepository.save(product);


// ĐÚNG - để cơ sở dữ liệu xử lý phép tính trực tiếp (atomic)

await productRepository.decrement({ id }, 'stock', 1);
```

5.4. Atomic Update

Khái niệm: Atomic Update là kỹ thuật cập nhật dữ liệu trực tiếp trong một câu lệnh SQL nhằm đảm bảo tính nguyên tử (Atomicity).

Thay vì:

Đọc dữ liệu



Kiểm tra



Cập nhật

Hệ thống sẽ thực hiện tất cả trong một câu lệnh SQL.

Ví dụ:

UPDATE products

SET stock = stock - 1

WHERE id = 1

AND stock > 0;

Nếu:

Stock = 0

Câu lệnh sẽ không cập nhật bất kỳ dòng nào.

Điều này giúp tránh Race Condition khi nhiều người cùng mua sản phẩm.

Ưu điểm

- Giảm Race Condition.
- Không cần đọc dữ liệu trước khi cập nhật.
- Tận dụng cơ chế khóa của Database.
- Hiệu năng tốt hơn cách Read → Update truyền thống.

Atomic Update và Transaction

Atomic Update và Transaction không thay thế cho nhau.

Atomic Update đảm bảo một câu lệnh SQL được thực hiện một cách nguyên tử.

Transaction đảm bảo nhiều câu lệnh SQL được thực hiện như một đơn vị công việc.

Ví dụ:

Transaction

|— Atomic Update Product

|— Insert Booking

|— Insert Payment

Trong thực tế, hai kỹ thuật này thường được kết hợp để đảm bảo tính nhất quán của dữ liệu.

5.5. Idempotency

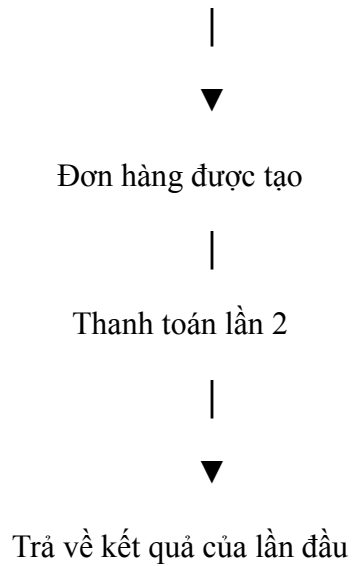
Khái niệm: Idempotency là tính chất mà một yêu cầu được gửi nhiều lần nhưng chỉ tạo ra một kết quả cuối cùng. Nói cách khác, dù Client gửi cùng một Request một hay nhiều lần thì trạng thái của hệ thống sau khi xử lý vẫn giống nhau.

Ví dụ:

Người dùng nhấn nút Thanh toán nhiều lần do mạng chậm.

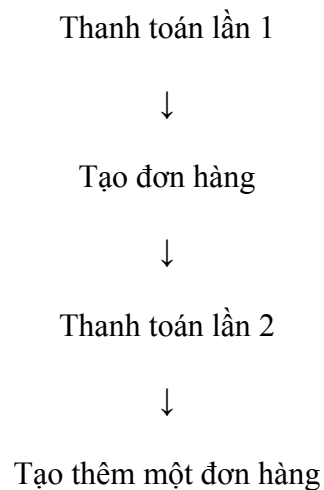
Nếu hệ thống hỗ trợ Idempotency:

Thanh toán lần 1



Hệ thống chỉ tạo một đơn hàng duy nhất.

Nếu không có Idempotency:



Điều này có thể gây:

- Thanh toán hai lần.
- Tạo nhiều đơn hàng.
- Đặt trùng vé.
- Đặt trùng phòng.

Khi nào cần Idempotency

Idempotency đặc biệt quan trọng đối với các thao tác có khả năng gây thay đổi dữ liệu.

Ví dụ:

- Thanh toán.
- Đặt vé.

- Đặt phòng.
- Tạo đơn hàng.
- Webhook.
- API của bên thứ ba.

Các API chỉ đọc dữ liệu như GET thường đã có tính Idempotent nên không cần triển khai thêm.

5.5.1. Cách hoạt động

Thông thường Client sẽ sinh một Idempotency Key và gửi kèm trong Request.

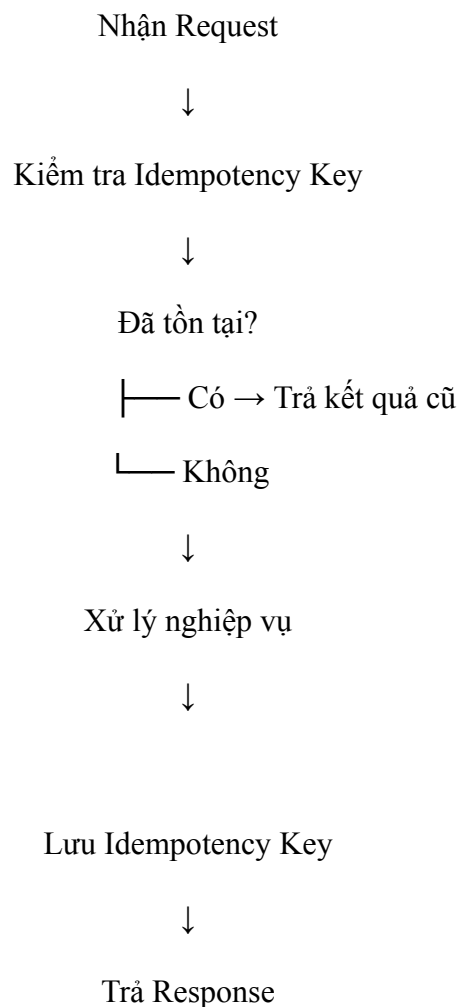
Ví dụ:

POST /payments

Idempotency-Key:

5b7d4e91-2ab1-4d5e-bb87-acde12345678

Backend thực hiện các bước:



Nhờ đó cùng một Request sẽ không được xử lý nhiều lần.

5.5.2. Idempotency Key

Idempotency Key là một mã định danh duy nhất (Unique Identifier) đại diện cho một yêu cầu của Client.

Thông thường sử dụng:

- UUID
- ULID
- Random String

Ví dụ:

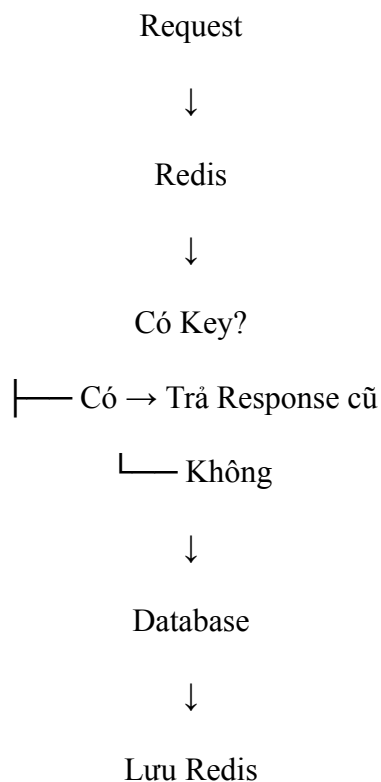
e70de5fd-fb2f-4bb8-a6d4-8cb91fd6d7ef

Mỗi thao tác chỉ sử dụng một Idempotency Key.

Nếu người dùng thực hiện một thao tác mới thì Client phải tạo một Key mới.

Lưu Idempotency Key ở đâu?

Trong các hệ thống thực tế, Idempotency Key thường được lưu trong Redis.



Redis được ưu tiên vì:

- Tốc độ truy cập rất nhanh.
- Hỗ trợ TTL.
- Phù hợp với dữ liệu tạm thời.

Một số hệ thống vẫn lưu thêm vào Database để phục vụ Audit hoặc đảm bảo an toàn khi Redis gặp sự cố.

Thời gian lưu

Idempotency Key không cần lưu vĩnh viễn.

Thông thường:

- 5 phút.
- 30 phút.
- 24 giờ.

Sau khoảng thời gian này, Key sẽ tự động hết hạn (TTL).

5.6. Retry

Khái niệm

Retry là kỹ thuật tự động thực hiện lại một thao tác khi thao tác đó thất bại do các lỗi tạm thời (Transient Error).

Mục tiêu của Retry là tăng khả năng thành công của hệ thống mà không cần người dùng hoặc lập trình viên thực hiện lại thao tác bằng tay.

Retry có thể được áp dụng trong nhiều tình huống khác nhau như:

- Gửi lại HTTP Request.
- Gọi API của bên thứ ba.
- Kết nối Redis.
- Kết nối Database.
- Gửi Email.
- Xử lý Queue Job.

Ví dụ:

Worker



Gửi Email



SMTP Server tạm thời không phản hồi



Retry



Gửi thành công

Trong ví dụ trên, Job chỉ thất bại do SMTP tạm thời gặp sự cố. Retry giúp hệ thống tự động gửi lại Email mà không cần người dùng thao tác lại.

5.6.1. Retryable Error

Retry chỉ nên áp dụng đối với các lỗi có khả năng được khắc phục sau một khoảng thời gian ngắn. Những lỗi này được gọi là Retryable Error.

Một số lỗi thường được Retry:

Lỗi	Có nên Retry
Network Error	✓
Connection Timeout	✓
502 Bad Gateway	✓
503 Service Unavailable	✓
504 Gateway Timeout	✓
Redis tạm thời mất kết nối	✓
RabbitMQ/BullMQ tạm thời không phản hồi	✓

Ngược lại, không nên Retry đối với các lỗi do dữ liệu hoặc quyền truy cập.

Lỗi	Có nên Retry
400 Bad Request	✗
401 Unauthorized	✗
403 Forbidden	✗
404 Not Found	✗

Validation Error	✗
------------------	---

Ví dụ, nếu người dùng gửi sai định dạng Email thì Retry sẽ không giải quyết được vấn đề.

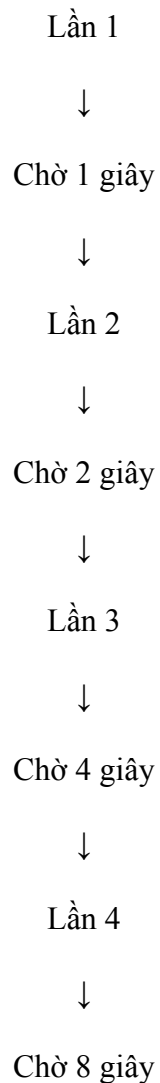
5.6.2. Exponential Backoff

Nếu Retry liên tục ngay sau khi gặp lỗi, hệ thống có thể tạo thêm áp lực lên Server, khiến Server càng khó phục hồi.

Để giải quyết vấn đề này, người ta sử dụng Exponential Backoff.

Exponential Backoff là kỹ thuật tăng dần khoảng thời gian chờ giữa các lần Retry theo cấp số nhân.

Ví dụ:



Trong thực tế, Exponential Backoff thường được kết hợp với Jitter (thêm một khoảng thời gian ngẫu nhiên) để tránh hiện tượng nhiều Client cùng Retry tại một thời điểm, gây quá tải cho hệ thống.

5.6.3. Max Retry

Max Retry là số lần tối đa hệ thống được phép Retry trước khi kết luận thao tác thất bại.

Ví dụ:

Retry tối đa 3 lần

↓

Lần 1

↓

Fail

↓

Lần 2

↓

Fail

↓

Lần 3

↓

Fail

↓

Đánh dấu Failed

Sau khi vượt quá số lần Retry, hệ thống sẽ thực hiện các hành động khác như:

- Trả lỗi cho Client.
- Ghi Log.
- Đưa Job vào Dead Letter Queue.
- Thông báo cho người quản trị.

Việc giới hạn số lần Retry giúp tránh vòng lặp vô hạn và hạn chế tiêu tốn tài nguyên hệ thống.

5.6.4. Lưu ý khi sử dụng Retry

Retry không phải lúc nào cũng phù hợp. Việc Retry không kiểm soát có thể làm tăng tải cho hệ thống hoặc tạo ra dữ liệu không nhất quán.

Khi triển khai Retry, cần lưu ý:

- Chỉ Retry đối với Retryable Error.
- Giới hạn số lần Retry bằng Max Retry.
- Sử dụng Exponential Backoff để giảm áp lực lên Server.
- Với các Request làm thay đổi dữ liệu (Create Order, Payment, Booking...), cần kết hợp Idempotency để tránh xử lý trùng lặp.
- Đối với Queue, sau khi vượt quá số lần Retry nên chuyển Job sang Dead Letter Queue (DLQ) để kiểm tra và xử lý sau.

5.7. Queue

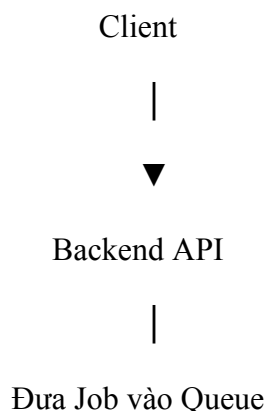
5.7.1. Khái niệm

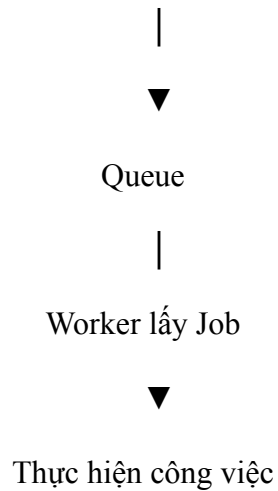
Trong một hệ thống Backend, không phải mọi công việc đều cần được xử lý ngay lập tức. Có những tác vụ mất nhiều thời gian hoặc không cần phản hồi ngay cho người dùng. Nếu xử lý trực tiếp trong luồng Request, thời gian phản hồi sẽ tăng lên, ảnh hưởng đến trải nghiệm người dùng và làm giảm khả năng chịu tải của hệ thống.

Để giải quyết vấn đề này, các hệ thống hiện đại thường sử dụng Queue (Hàng đợi).

Queue là cơ chế lưu trữ tạm thời các công việc (Job) và xử lý chúng theo cơ chế bất đồng bộ (Asynchronous Processing). Khi Client gửi một yêu cầu, Backend chỉ cần đưa công việc vào Queue và phản hồi ngay. Sau đó, một tiến trình khác (Worker) sẽ lấy công việc từ Queue để xử lý.

Luồng hoạt động của Queue:





Nhờ đó, thời gian phản hồi của API được rút ngắn đáng kể.

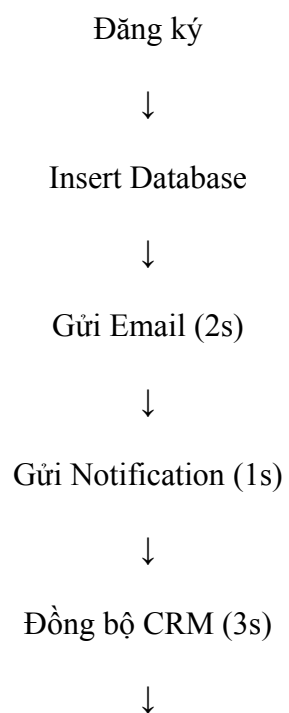
Tại sao cần Queue?

Giả sử người dùng đăng ký tài khoản.

Sau khi đăng ký thành công, hệ thống cần:

- Lưu người dùng vào Database.
- Gửi Email xác nhận.
- Gửi thông báo.
- Đồng bộ dữ liệu sang CRM.

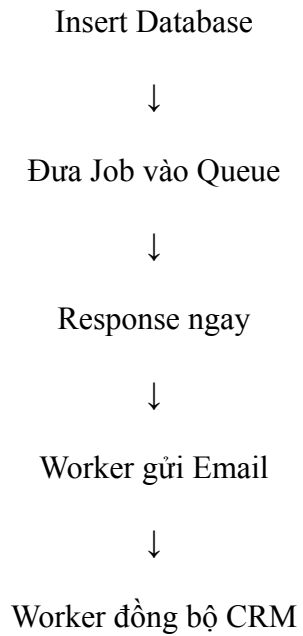
Nếu thực hiện tuần tự:



Response

Người dùng phải chờ khoảng 6 giây.

Nếu sử dụng Queue:



Người dùng chỉ cần chờ khoảng 100–200ms.

Khi nào nên sử dụng Queue?

Queue thường được sử dụng cho các tác vụ:

- Gửi Email.
- Gửi SMS.
- Gửi Notification.
- Resize ảnh.
- Upload Video.
- AI Processing.
- Sinh báo cáo.
- Đồng bộ dữ liệu.
- Xuất Excel.
- Import dữ liệu.
- Gửi Webhook.

Các tác vụ này đều có đặc điểm:

- Mất nhiều thời gian.
- Không cần phản hồi ngay.
- Có thể xử lý sau.

Ưu điểm - Nhược điểm

Ưu:

- Giảm thời gian phản hồi của API.
- Giảm tải cho Backend.
- Dễ mở rộng hệ thống.
- Có thể Retry khi thất bại.
- Xử lý song song nhiều Job.

Nhược:

- Kiến trúc phức tạp hơn.
- Cần thêm Redis hoặc Message Broker.
- Khó Debug hơn xử lý đồng bộ.

5.7.2. BullMQ

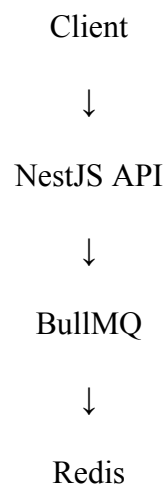
BullMQ là thư viện Queue phổ biến trong hệ sinh thái Node.js, được xây dựng trên Redis.

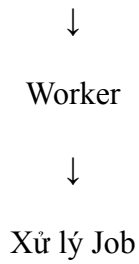
BullMQ giúp lập trình viên:

- Tạo Queue.
- Thêm Job.
- Retry Job.
- Delay Job.
- Lập lịch Job.
- Theo dõi trạng thái Job.

Trong NestJS, BullMQ thường được sử dụng kết hợp với Redis để xử lý các tác vụ nền (Background Job).

Cơ chế hoạt động





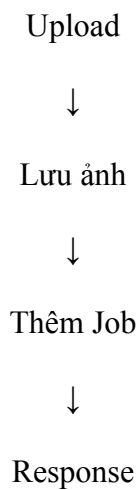
Redis đóng vai trò lưu trữ Queue.

Worker liên tục theo dõi Queue.

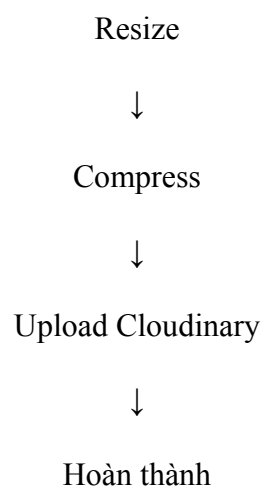
Khi có Job mới, Worker sẽ lấy Job ra và xử lý.

Ví dụ

Người dùng upload ảnh.



Worker:



Người dùng không phải chờ quá trình resize.

```
@Injectable()

export class EmailProducer {

  constructor(@InjectQueue('email') private emailQueue: Queue) {}

  async sendWelcomeEmail(userId: number) {

    await this.emailQueue.add(

      'welcome-email',

      { userId },

      { attempts: 3, backoff: { type: 'exponential', delay: 2000 } }, // Retry

    );

  }

}

@Processor('email')

export class EmailConsumer extends WorkerHost {

  async process(job: Job) {

    // Nếu process() ném lỗi liên tục quá số 'attempts',

    // job sẽ được chuyển sang trạng thái "failed" (đóng vai trò Dead Letter Queue)

    await sendEmail(job.data.userId);

  }

}
```

5.7.3 Job

Job là một đơn vị công việc được đưa vào Queue.

Mỗi Job chứa:

- Tên Job.
- Dữ liệu cần xử lý.
- Trạng thái.
- Thời gian tạo.
- Số lần Retry.

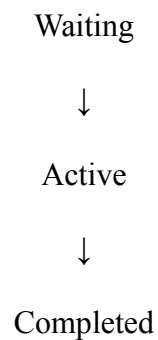
Ví dụ:

```
{  
  "name": "send-email",  
  "email": "abc@gmail.com"  
}
```

Worker sẽ đọc Job này và thực hiện việc gửi Email.

Vòng đời của Job

Một Job thường trải qua các trạng thái:



Nếu lỗi:



Failed



Retry

Nếu Retry vẫn thất bại:

Failed



Dead Letter Queue

5.7.4 Retry trong Queue

Trong thực tế, nhiều Job thất bại chỉ do lỗi tạm thời.

Ví dụ:

- SMTP bị chậm.
- API thanh toán quá tải.
- Redis mất kết nối vài giây.

Nếu hủy Job ngay sẽ làm mất dữ liệu.

Do đó Queue thường hỗ trợ Retry tự động.

Ví dụ:

Send Email



SMTP lỗi



Retry



Gửi thành công

Lợi ích

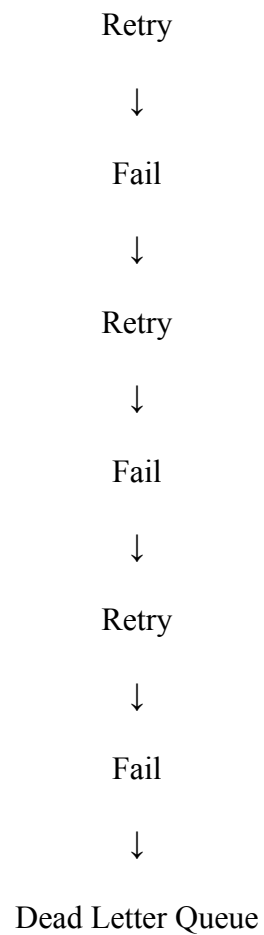
- Tăng tỷ lệ thành công.
- Không cần người quản trị xử lý thủ công.

- Giảm mất dữ liệu.

5.7.5 Dead Letter Queue (DLQ)

Dead Letter Queue là nơi lưu các Job đã Retry nhiều lần nhưng vẫn thất bại.

Ví dụ:

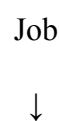


Các Job trong DLQ sẽ được:

- Kiểm tra thủ công.
- Sửa lỗi.
- Chạy lại nếu cần.

Tại sao cần DLQ?

Nếu không có DLQ:



Fail



Mất luôn

Hệ thống sẽ mất dữ liệu.

DLQ giúp đảm bảo mọi Job đều được theo dõi.

5.8. Cron Job (Tác vụ theo lịch)

Khái niệm: Cron Job (còn gọi là Scheduled Job) là kỹ thuật lập lịch cho hệ thống tự động chạy một tác vụ nhất định theo một chu kỳ hoặc thời điểm định trước, mà không cần có sự kiện nào kích hoạt từ bên ngoài. Tên gọi "cron" bắt nguồn từ tiện ích lập lịch lâu đời trên hệ điều hành Unix/Linux, sử dụng cú pháp biểu thức thời gian gọi là **cron expression** để mô tả lịch chạy (ví dụ: chạy vào lúc 0 giờ mỗi ngày, hoặc mỗi 15 phút một lần).

Một số ví dụ nghiệp vụ thường dùng Cron Job: dọn dẹp dữ liệu tạm/hết hạn mỗi đêm, gửi email báo cáo định kỳ mỗi tuần, đồng bộ dữ liệu với hệ thống bên ngoài mỗi giờ, tự động hủy các đơn hàng chưa thanh toán sau một khoảng thời gian.

Phân biệt Cron Job và Queue: Đây là hai kỹ thuật thường bị nhầm lẫn vì cùng thuộc nhóm "xử lý tác vụ nền" (background processing), nhưng khác nhau ở **điều gì kích hoạt việc chạy**:

Tiêu chí	Cron Job	Queue
Cơ chế kích hoạt	Theo thời gian (time-driven) — đến giờ là tự chạy	Theo sự kiện (event-driven) — có việc mới thì mới xử lý
Ví dụ	Mỗi đêm quét và xóa file tạm đã hết hạn	Ngay khi có đơn hàng mới, đẩy job gửi email xác nhận
Đặc điểm	Chạy định kỳ, kể cả khi không có dữ liệu cần xử lý	Chỉ chạy khi có Job được đưa vào hàng đợi

Một số lưu ý khi triển khai Cron Job trong hệ thống thực tế:

- **Tránh chạy trùng lặp (Overlapping):** Nếu một tác vụ chạy lâu hơn chu kỳ lập lịch (ví dụ tác vụ chạy 20 phút nhưng lịch đặt chạy mỗi 15 phút), cần có cơ chế khóa (tương tự Distributed Lock ở mục 3.3) để tránh hai lần chạy chồng lên nhau.
- **Chạy trên nhiều instance:** Khi hệ thống Backend chạy nhiều bản sao (instance) cùng lúc để chịu tải, cần đảm bảo Cron Job chỉ được thực thi bởi **đúng một instance** tại một thời điểm, tránh tình trạng cùng một tác vụ (ví dụ gửi email báo cáo) bị chạy lặp nhiều lần.

typescript

```
// NestJS - Cron Job với @nestjs/schedule

import { Cron, CronExpression } from '@nestjs/schedule';

@Injectable()

export class OrderCleanupService {

  @Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT) // chạy lúc 0h mỗi ngày

  async cancelUnpaidOrders() {

    await this.orderRepository.update(

      { status: 'pending', createdAt: LessThan(oneDayAgo()) },

      { status: 'cancelled' },

    );

  }

  @Cron('*/15 * * * *') // cron expression: chạy mỗi 15 phút

  async syncExternalData() {

    await this.syncService.sync();

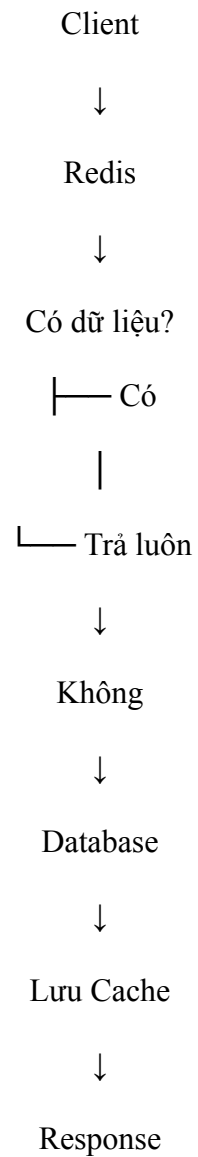
  }

}
```

5.9. Cache

Khái niệm: Cache là kỹ thuật lưu tạm dữ liệu đã được truy vấn hoặc tính toán ở một nơi có tốc độ truy xuất nhanh (thường là bộ nhớ RAM, ví dụ Redis), nhằm giảm số lần truy vấn trực tiếp đến cơ sở dữ liệu, giúp hệ thống phản hồi nhanh hơn.

Ví dụ:



Thay vì đọc Database mỗi lần, Backend ưu tiên đọc từ Cache.

Tại sao cần Cache?

Giả sử:

`SELECT * FROM products`

Database mất:

100ms

Nếu dùng Redis:

2ms

Hiệu năng tăng lên đáng kể.

Khi nào nên Cache?

Các dữ liệu:

- Ít thay đổi.
- Đọc nhiều.
- Truy vấn chậm.

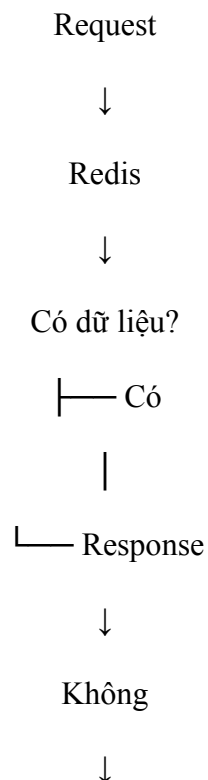
Ví dụ:

- Danh mục sản phẩm.
- Danh sách tỉnh thành.
- Danh sách phim.
- Hồ sơ người dùng.
- Trang chủ.

5.8.1 Cache Aside Pattern

Cache Aside là chiến lược Cache phổ biến nhất — khi đọc dữ liệu, ứng dụng kiểm tra cache trước; nếu không có (cache miss) thì mới truy vấn cơ sở dữ liệu, sau đó lưu kết quả vào cache để lần sau dùng lại.

Quy trình:



Database



Lưu Redis



Response

Lần đầu: Database

Các lần sau: Redis

```
// NestJS - Cache Aside Pattern

async getProduct(id: number) {

  const cacheKey = `product:${id}`;

  const cached = await this.cacheManager.get(cacheKey);

  if (cached) return cached; // cache hit

  const product = await this.productRepository.findOne({ where: { id } });

  await this.cacheManager.set(cacheKey, product, { ttl: 300 }); // TTL 5 phút

  return product;
}

async updateProduct(id: number, data: any) {

  const product = await this.productRepository.save({ id, ...data });

  await this.cacheManager.del(`product:${id}`); // Cache Invalidation

  return product;
}
```

5.8.2. TTL (Time To Live)

Thời gian sống của một dữ liệu trong cache; sau khoảng thời gian này, dữ liệu tự động bị xóa để tránh dữ liệu cũ tồn tại quá lâu.

Tại sao cần TTL?

Nếu không có TTL: Database đã cập nhật mà Redis vẫn giữ dữ liệu cũ

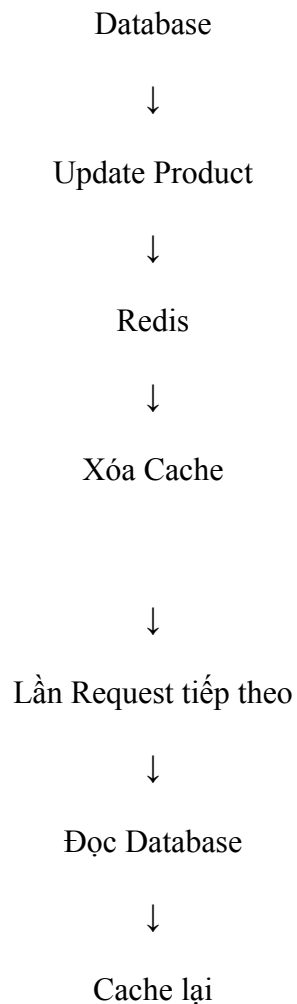
Người dùng sẽ nhìn thấy dữ liệu không chính xác.

TTL giúp dữ liệu được làm mới theo thời gian.

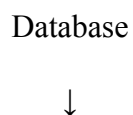
5.8.3. Cache Invalidation

Là hành động chủ động xóa hoặc cập nhật lại dữ liệu trong cache khi dữ liệu gốc trong cơ sở dữ liệu thay đổi, tránh tình trạng cache trả về dữ liệu cũ không còn chính xác.

Ví dụ:



Nếu không xóa Cache:



Đã cập nhật



Redis



Vẫn dữ liệu cũ

Hệ thống sẽ trả về dữ liệu sai.

Các cách Invalidate Cache

Thông dụng nhất gồm:

- Xóa Cache ngay sau khi cập nhật Database.
- Sử dụng TTL.
- Cập nhật lại Cache sau khi ghi Database (Write Through).

Trong các dự án Backend thông thường, **xóa Cache sau khi cập nhật Database kết hợp với TTL** là giải pháp được sử dụng phổ biến vì đơn giản và hiệu quả.

5.10. Rate Limiting

Là kỹ thuật giới hạn số lượng request mà một client (theo IP, theo tài khoản, theo API key...) được phép gửi đến hệ thống trong một khoảng thời gian nhất định, nhằm bảo vệ hệ thống khỏi bị quá tải hoặc bị lạm dụng (ví dụ: tấn công brute-force, spam request).

```
// NestJS - sử dụng @nestjs/throttler

@Throttle({ default: { limit: 10, ttl: 60000 } }) // tối đa 10 request / 60 giây

@Get('search')

search(@Query('q') q: string) {

  return this.searchService.search(q);

}
```

5.11 Timeout

Là cơ chế đặt ra một khoảng thời gian tối đa cho phép một thao tác (gọi API bên ngoài, truy vấn cơ sở dữ liệu...) được thực thi. Nếu quá thời gian này mà chưa có kết quả, hệ thống sẽ chủ động hủy thao tác và trả về lỗi, thay vì để request "treo" vô thời hạn, gây tắc nghẽn tài nguyên hệ thống.

typescript

```
// Express.js - timeout khi gọi API bên ngoài

const response = await axios.get('https://api.example.com/data', {
  timeout: 5000, // hủy nếu quá 5 giây chưa có phản hồi
});
```

5.12. Circuit Breaker

Circuit Breaker là kỹ thuật giúp hệ thống "tạm ngừng" gọi đến một dịch vụ đang gặp sự cố (liên tục lỗi hoặc timeout), thay vì tiếp tục gửi request và chờ thất bại nhiều lần. Cơ chế này hoạt động tương tự cầu dao điện, gồm ba trạng thái:

- **Closed:** Hoạt động bình thường, request được gửi đi như thường lệ.
- **Open:** Sau khi phát hiện quá nhiều lỗi liên tiếp, "ngắt mạch" — từ chối gửi request đến dịch vụ đó trong một khoảng thời gian, trả về lỗi ngay lập tức để tránh lãng phí tài nguyên chờ đợi.
- **Half-Open:** Sau khoảng thời gian ngắt, cho phép một số ít request thử nghiệm đi qua để kiểm tra dịch vụ đã hồi phục hay chưa, trước khi quyết định đóng lại mạch (Closed) hoàn toàn.

5.13. Logging

Logging là việc ghi lại các sự kiện xảy ra trong quá trình hệ thống vận hành, phục vụ mục đích theo dõi, gỡ lỗi và phân tích sau này.

- **Structured Logging (Ghi log có cấu trúc):** Thay vì ghi log dưới dạng chuỗi văn bản tự do, dữ liệu log được ghi theo định dạng có cấu trúc (thường là JSON), giúp dễ dàng tìm kiếm, lọc và phân tích bằng công cụ tự động.
- **Request ID / Correlation ID:** Một mã định danh duy nhất được gắn cho mỗi request ngay khi vào hệ thống, đi xuyên suốt qua tất cả các service liên quan xử lý request đó. Nhờ vậy, khi cần truy vết một lỗi cụ thể, có thể lọc toàn bộ log liên quan đến một request duy nhất, kể cả khi hệ thống gồm nhiều service khác nhau.

```
// Express.js - middleware gắn Correlation ID

app.use((req, res, next) => {

  req.correlationId = req.headers['x-correlation-id'] || crypto.randomUUID();

  res.setHeader('x-correlation-id', req.correlationId);

  next();
});
```

```
});

// Structured logging
logger.info({
  correlationId: req.correlationId,
  action: 'CREATE_ORDER',
  userId: req.user.id,
  timestamp: new Date().toISOString(),
});
```

5.14. Error Handling

Là việc thiết kế cách hệ thống phát hiện, phân loại và phản hồi lại các lỗi phát sinh trong quá trình xử lý, đảm bảo lỗi được xử lý nhất quán và không làm lộ thông tin nhạy cảm của hệ thống (ví dụ: stack trace, thông tin cơ sở dữ liệu) ra ngoài cho người dùng.

```
// NestJS - Global Exception Filter

@Catch()

export class AllExceptionsFilter implements ExceptionFilter {

  catch(exception: unknown, host: ArgumentsHost) {

    const ctx = host.switchToHttp();

    const response = ctx.getResponse();

    const status =

      exception instanceof HttpException ? exception.getStatus() : 500;

    response.status(status).json({

      statusCode: status,

      message: exception instanceof HttpException

        ? exception.message

        : 'Lỗi hệ thống, vui lòng thử lại sau',
```

```
});  
  
}  
  
}
```

5.15. Monitoring Monitoring là quá trình theo dõi liên tục tình trạng hoạt động của hệ thống trong thực tế, giúp phát hiện sớm sự cố trước khi ảnh hưởng nghiêm trọng đến người dùng.

- **Health Check (Kiểm tra tình trạng hoạt động):** Một endpoint đơn giản (thường là GET /health) cho phép hệ thống giám sát bên ngoài (hoặc hạ tầng như Kubernetes) kiểm tra định kỳ xem service có đang hoạt động bình thường hay không.
- **Metrics (Chỉ số đo lường):** Các số liệu định lượng về tình trạng hệ thống được thu thập theo thời gian, ví dụ: số lượng request/giây, thời gian phản hồi trung bình, tỉ lệ lỗi — thường được trực quan hóa bằng các công cụ như Prometheus, Grafana.

Logging (mục 5.14) cũng là một thành phần quan trọng của Monitoring nói chung, vì log chi tiết là nguồn dữ liệu để phân tích khi có cảnh báo bất thường từ Metrics.

typescript

```
// NestJS - Health Check đơn giản  
  
@Get('health')  
  
healthCheck() {  
  
    return { status: 'ok', timestamp: new Date().toISOString() };  
  
}
```

CHƯƠNG 6. SECURITY

Bảo mật là yếu tố bắt buộc phải cân nhắc trong mọi hệ thống Backend, vì Backend là nơi trực tiếp xử lý dữ liệu người dùng và nghiệp vụ quan trọng. Nội dung chương này được chia thành ba nhóm:

- (1) Xác thực & phân quyền — xác định "ai đang truy cập" và "họ được phép làm gì";
- (2) Phòng chống các hình thức tấn công phổ biến trên ứng dụng web;
- (3) Bảo mật tầng vận chuyển & giới hạn truy cập.

6.1. Authentication

Khái niệm: Authentication là quá trình xác minh danh tính của người dùng — trả lời cho câu hỏi "**bạn là ai?**". Hệ thống cần chắc chắn rằng người đang gửi request thực sự là người họ tự nhận, trước khi cho phép thực hiện bất kỳ điều gì tiếp theo.

Các yếu tố xác thực (Authentication Factors): Xác thực thường dựa trên một hoặc nhiều trong ba loại yếu tố sau:

Loại yếu tố	Mô tả	Ví dụ
Something you know (điều bạn biết)	Thông tin bí mật chỉ người dùng biết	Mật khẩu, mã PIN
Something you have (thứ bạn sở hữu)	Vật/thiết bị chỉ người dùng có	Điện thoại nhận OTP, khóa bảo mật vật lý (security key)
Something you are (đặc điểm của bạn)	Đặc điểm sinh trắc học duy nhất	Vân tay, khuôn mặt

Khi hệ thống yêu cầu kết hợp từ hai yếu tố trở lên (ví dụ: mật khẩu + mã OTP gửi về điện thoại), ta gọi đó là **Xác thực đa yếu tố (Multi-Factor Authentication - MFA)**, giúp tăng độ an toàn đáng kể — kẻ tấn công dù có được mật khẩu vẫn không thể đăng nhập nếu thiếu yếu tố thứ hai.

Hai cách tiếp cận phổ biến để duy trì trạng thái đăng nhập:

Cách tiếp cận	Cách hoạt động	Ưu điểm	Nhược điểm
Session-based (dựa trên phiên)	Server tạo một session sau khi đăng nhập, lưu trạng thái ở server (thường trong bộ nhớ hoặc Redis), gửi về client một session ID lưu trong cookie.	Dễ thu hồi (chỉ cần xóa session ở server); phù hợp ứng dụng web truyền thống.	Server phải lưu trạng thái (stateful), khó mở rộng theo chiều ngang nếu không đồng bộ session giữa các server.
Token-based (dựa trên token, ví dụ JWT)	Server không lưu trạng thái, mọi thông tin xác thực được mã hóa trong token, client tự giữ và gửi kèm mỗi request.	Server không cần lưu trạng thái (stateless), dễ mở rộng, phù hợp hệ thống nhiều service (microservices).	Khó thu hồi giữa chừng khi token chưa hết hạn (xem thêm mục 6.3 — Refresh Token).

Phần lớn các hệ thống Backend hiện đại, đặc biệt là hệ thống có API phục vụ nhiều nền tảng (web, mobile, bên thứ ba), thường ưu tiên hướng tiếp cận token-based, cụ thể là JWT — sẽ được trình bày ở mục 6.3.

6.2. Authorization

Khái niệm: Authorization là quá trình xác định người dùng **đã được xác thực** có quyền thực hiện một hành động cụ thể hay không — trả lời cho câu hỏi "**bạn được phép làm gì?**". Authentication luôn diễn ra trước Authorization: hệ thống phải biết người dùng là ai trước khi quyết định họ được phép truy cập tài nguyên nào.

Ví dụ phân biệt: Đăng nhập thành công vào hệ thống là **Authentication**. Sau khi đăng nhập, việc hệ thống kiểm tra xem người dùng có phải là "Admin" để cho phép xóa một bài viết hay không là **Authorization**.

Vị trí kiểm tra Authorization: Việc kiểm tra quyền hạn nên được thực hiện ở tầng Backend (thường tại middleware/guard hoặc ngay trong service xử lý nghiệp vụ), **không bao giờ chỉ dựa vào việc ấn nút bấm trên giao diện**. Vì client (trình duyệt, app di động) hoàn toàn nằm ngoài tầm kiểm soát của hệ thống, người dùng có kỹ thuật vẫn có thể tự gửi request trực tiếp đến API mà bỏ qua giao diện, nên mọi quy tắc phân quyền quan trọng bắt buộc phải được xác thực lại ở phía server.

Hai mô hình phân quyền phổ biến nhất là **RBAC** (trình bày ở mục 6.4) và **ABAC (Attribute-Based Access Control)** — mô hình cấp quyền linh hoạt hơn, dựa trên nhiều thuộc tính kết hợp (ví dụ: chỉ cho phép sửa đơn hàng nếu người dùng là chủ đơn hàng đó và đơn hàng chưa được giao). ABAC mạnh mẽ hơn RBAC nhưng cũng phức tạp hơn để triển khai và quản lý.

6.3. JWT

6.3.1. Giới thiệu JWT

Khái niệm: JWT là một định dạng token phổ biến, được chuẩn hóa theo RFC 7519, dùng để truyền tải thông tin xác thực giữa client và server một cách an toàn, gói gọn trong một chuỗi văn bản duy nhất.

Cấu trúc: JWT gồm 3 phần, ngăn cách bởi dấu chấm: Header.Payload.Signature

- **Header:** Chứa thông tin về loại token và thuật toán mã hóa chữ ký được sử dụng (ví dụ HS256, RS256).
- **Payload:** Chứa dữ liệu thực tế (gọi là các "claims"), ví dụ userId, role, thời điểm hết hạn (exp). **Lưu ý quan trọng:** phần Payload chỉ được **mã hóa Base64** (encode) chứ không phải **mã hóa bảo mật** (encrypt) — nghĩa là bất kỳ ai cũng có thể giải mã và đọc được nội dung bên trong, dù không thể chỉnh sửa nó (nhờ Signature). Vì vậy, không nên lưu thông tin nhạy cảm (mật khẩu, số thẻ...) trong Payload của JWT.
- **Signature:** Chữ ký được tạo ra từ Header, Payload và một khóa bí mật (secret key), giúp server xác minh token không bị chỉnh sửa sau khi phát hành.

Hai họ thuật toán ký phổ biến:

Thuật toán	Cách hoạt động	Phù hợp khi
HS256 (đối xứng)	Dùng một khóa bí mật duy nhất để vừa ký vừa xác minh token.	Hệ thống chỉ có một Backend duy nhất tự phát hành và tự xác minh token.
RS256 (bất đối xứng)	Dùng cặp khóa riêng tư (private key) để ký và khóa công khai (public key) để xác minh.	Hệ thống gồm nhiều service (microservices) — chỉ nơi phát hành token cần giữ private key, các service khác chỉ cần public key để xác minh mà không thể tự tạo token giả.

Luồng hoạt động: Sau khi đăng nhập thành công, server tạo JWT và trả về cho client; các request sau đó, client gửi kèm JWT (thường trong header Authorization: Bearer <token>) để server xác thực mà không cần truy vấn lại cơ sở dữ liệu để kiểm tra phiên đăng nhập (đặc tính stateless).

Nơi lưu trữ JWT ở phía client và rủi ro liên quan:

Nơi lưu trữ	Rủi ro chính
localStorage	Dễ bị đánh cắp bởi tấn công XSS (mục 6.10), vì bất kỳ đoạn script nào chạy trên trang đều đọc được.
HttpOnly Cookie	An toàn hơn trước XSS (JavaScript không đọc được cookie loại này), nhưng cần phòng chống CSRF (mục 6.9) vì cookie được trình duyệt tự động gửi kèm mỗi request.

typescript

```
// NestJS - tạo và xác thực JWT

@Injectable()
export class AuthService {

  constructor(private jwtService: JwtService) {}

  login(user: User) {

    const payload = { sub: user.id, email: user.email, role: user.role };

    return {

      accessToken: this.jwtService.sign(payload, { expiresIn: '15m' }),

    };

  }

}

// Guard kiểm tra JWT trên các route cần bảo vệ

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

```

@UseGuards (JwtAuthGuard)

@Get ('profile')

getProfile (@Req () req) {

    return req.user;

}

```

6.3.2 Refresh Token

Khái niệm: Vì lý do bảo mật, Access Token (JWT) thường được cấu hình thời gian sống rất ngắn (ví dụ 15 phút) — nếu token bị đánh cắp, kẻ tấn công cũng chỉ có thể lạm dụng trong khoảng thời gian ngắn đó. Refresh Token là một token riêng, có thời gian sống dài hơn nhiều (ví dụ 7-30 ngày), dùng để xin cấp lại Access Token mới khi Access Token cũ hết hạn — giúp người dùng không phải đăng nhập lại liên tục nhưng vẫn giới hạn được thời gian hiệu lực của Access Token.

Vấn đề thu hồi (Revocation): Vì JWT là stateless, server thường không có cách nào "hủy" một Access Token đã phát hành trước khi nó tự hết hạn. Do đó, Refresh Token thường được lưu trạng thái ở phía server (trong cơ sở dữ liệu hoặc Redis), để khi cần (ví dụ người dùng bấm "đăng xuất trên mọi thiết bị", hoặc phát hiện tài khoản bị xâm nhập), hệ thống có thể **thu hồi Refresh Token** đó — khiến kẻ tấn công không thể xin cấp Access Token mới, dù Access Token cũ vẫn còn hiệu lực trong ít phút còn lại.

Refresh Token Rotation: Là kỹ thuật nâng cao, mỗi lần Refresh Token được dùng để xin Access Token mới, hệ thống sẽ đồng thời cấp luôn một Refresh Token mới và vô hiệu hóa Refresh Token cũ. Nếu một Refresh Token đã bị vô hiệu hóa nhưng vẫn có người cố dùng lại (dấu hiệu bị đánh cắp), hệ thống có thể lập tức thu hồi toàn bộ chuỗi token liên quan để ngăn chặn.

typescript

```

// NestJS - endpoint cấp lại Access Token từ Refresh Token

@Post ('refresh')

async refresh (@Body ('refreshToken') refreshToken: string) {

    const payload = this.jwtService.verify (refreshToken, { secret:
process.env.REFRESH_SECRET });

    // Kiểm tra Refresh Token có còn hợp lệ trong hệ thống (chưa bị thu hồi) hay
không

```



```

const isValid = await this.refreshTokenRepository.exists(refreshToken);

if (!isValid) throw new UnauthorizedException('Refresh token không hợp lệ');

const newAccessToken = this.jwtService.sign(

  { sub: payload.sub, email: payload.email },

  { expiresIn: '15m' },

);

return { accessToken: newAccessToken };
}

```

6.4. RBAC

Khái niệm: RBAC là mô hình phân quyền dựa trên vai trò (role). Thay vì gán quyền trực tiếp cho từng người dùng, hệ thống gán người dùng vào một hoặc nhiều vai trò (ví dụ: Admin, Editor, User), và quyền hạn được định nghĩa theo vai trò đó.

Các thành phần cơ bản của RBAC:

- **Role (Vai trò):** Một nhóm quyền hạn được đặt tên, ví dụ "Admin", "Editor".
- **Permission (Quyền hạn):** Một hành động cụ thể được phép thực hiện, ví dụ post:delete, user:view.
- **User-Role Assignment:** Việc gán một hoặc nhiều vai trò cho người dùng.

Ưu điểm: Dễ quản lý khi số lượng người dùng lớn — chỉ cần thay đổi quyền của một vai trò, mọi người dùng thuộc vai trò đó tự động được cập nhật quyền, không cần chỉnh sửa từng tài khoản.

Hạn chế: RBAC không xử lý tốt các quy tắc phân quyền phụ thuộc vào ngữ cảnh cụ thể (ví dụ: "chỉ cho phép sửa bài viết do chính mình tạo") — trường hợp này cần kết hợp thêm kiểm tra logic riêng trong code, hoặc chuyển sang mô hình ABAC linh hoạt hơn đã nhắc ở mục 6.2.

typescript

```

// NestJS - Guard kiểm tra Role

@Roles('admin')

@UseGuards(JwtAuthGuard, RolesGuard)

```

```

@Delete('posts/:id')

deletePost(@Param('id') id: string) {

    return this.postService.delete(id);

}

// RolesGuard

@Injectable()

export class RolesGuard implements CanActivate {

    constructor(private reflector: Reflector) {}

    canActivate(context: ExecutionContext): boolean {

        const requiredRoles = this.reflector.get<string[]>('roles',
context.getHandler());

        const { user } = context.switchToHttp().getRequest();

        return requiredRoles.includes(user.role);

    }

}

```

6.5. OAuth2

Khái niệm: OAuth2 là một giao thức chuẩn (được định nghĩa trong RFC 6749) cho phép một ứng dụng truy cập tài nguyên của người dùng trên một dịch vụ khác (ví dụ: Google, Facebook, GitHub) mà **không cần biết mật khẩu** của người dùng trên dịch vụ đó. Đây là cơ chế nền tảng cho tính năng "Đăng nhập bằng Google/Facebook" rất phổ biến hiện nay.

Các vai trò trong OAuth2:

- **Resource Owner:** Người dùng — chủ sở hữu dữ liệu.
- **Client:** Ứng dụng muốn truy cập dữ liệu của người dùng (ví dụ website của bạn).
- **Authorization Server:** Máy chủ cấp quyền (ví dụ máy chủ của Google) — nơi người dùng đăng nhập và đồng ý cấp quyền.
- **Resource Server:** Máy chủ chứa dữ liệu thực tế (ví dụ Google API trả về thông tin hồ sơ).

Luồng phổ biến nhất — Authorization Code Flow:

1. Người dùng được chuyển hướng đến trang đăng nhập của bên thứ ba (ví dụ Google).

2. Sau khi xác thực và đồng ý cấp quyền thành công, bên thứ ba chuyển hướng người dùng về lại ứng dụng, kèm theo một **mã ủy quyền tạm thời** (authorization code).
3. Ứng dụng (ở phía Backend, không phải trình duyệt) dùng mã này kèm theo client secret để đổi lấy **Access Token** — bước này diễn ra ở server nên client secret không bị lộ ra ngoài.
4. Ứng dụng dùng Access Token đó gọi API của bên thứ ba để lấy thông tin người dùng (email, tên...).

Phân biệt với OpenID Connect (OIDC): OAuth2 vốn được thiết kế cho mục đích **ủy quyền** (authorization — cho phép truy cập tài nguyên), không phải để **xác thực** (authentication — xác minh danh tính). OpenID Connect là một lớp mở rộng xây dựng trên nền OAuth2, bổ sung thêm token định danh (**ID Token**) chuẩn hóa, giúp việc "Đăng nhập bằng Google" trở nên đúng chuẩn xác thực hơn là chỉ đơn thuần ủy quyền.

```
// NestJS - sử dụng Passport Google Strategy (khái quát)

@Injectable()

export class GoogleStrategy extends PassportStrategy(Strategy, 'google') {

  constructor() {

    super({

      clientID: process.env.GOOGLE_CLIENT_ID,

      clientSecret: process.env.GOOGLE_CLIENT_SECRET,

      callbackURL: '/auth/google/callback',

      scope: ['email', 'profile'],

    });

  }

  async validate(accessToken: string, refreshToken: string, profile: any) {

    return { email: profile.emails[0].value, name: profile.displayName };

  }

}
```

6.6. Password Hashing (bcrypt/argon2)

Khái niệm: Mật khẩu người dùng **tuyệt đối không được lưu trực tiếp** dưới dạng văn bản thường (plain text) trong cơ sở dữ liệu — nếu cơ sở dữ liệu chẳng may bị rò rỉ, toàn bộ mật khẩu người dùng sẽ bị lộ ngay lập tức. Thay vào đó, mật khẩu cần được băm (hash) qua một thuật toán một chiều trước khi lưu trữ: có thể tính ra chuỗi hash từ mật khẩu gốc, nhưng gần như không thể tính ngược lại mật khẩu gốc từ chuỗi hash.

Salt là gì và tại sao cần thiết: Nếu chỉ băm đơn thuần, hai người dùng có cùng mật khẩu ("123456") sẽ cho ra cùng một chuỗi hash — kẻ tấn công có thể dùng một "bảng tra sẵn" các hash phổ biến (gọi là **Rainbow Table**) để dò ngược ra mật khẩu gốc hàng loạt. **Salt** là một chuỗi ngẫu nhiên được thêm vào mật khẩu trước khi băm, khiến cùng một mật khẩu ở hai tài khoản khác nhau cho ra hai chuỗi hash hoàn toàn khác nhau, vô hiệu hóa hoàn toàn kiểu tấn công bằng bảng tra sẵn.

So sánh các thuật toán băm mật khẩu phổ biến:

Thuật toán	Đặc điểm
bcrypt	Thuật toán lâu đời, được kiểm chứng rộng rãi, tự động sinh salt ngẫu nhiên cho mỗi lần băm. Có tham số "cost factor" (độ phức tạp) để tùy chỉnh mức độ tốn thời gian tính toán, nhằm gây khó khăn cho tấn công brute-force.
argon2	Thuật toán hiện đại hơn, từng thắng giải Password Hashing Competition (2015). Cho phép tùy chỉnh cả mức tiêu tốn bộ nhớ lẫn thời gian tính toán, khiến việc tấn công bằng phần cứng chuyên dụng (GPU, ASIC) trở nên tốn kém hơn nhiều so với bcrypt. Hiện được khuyến nghị là lựa chọn hàng đầu cho hệ thống mới.
scrypt	Tương tự argon2 ở việc tốn nhiều bộ nhớ để chống tấn công phần cứng, ra đời trước argon2, vẫn được sử dụng nhưng ít phổ biến hơn hai lựa chọn trên trong các dự án mới.

Lưu ý: Các thuật toán băm dữ liệu thông thường như MD5, SHA-1, SHA-256 **không phù hợp** để băm mật khẩu, vì chúng được thiết kế để chạy **càng nhanh càng tốt** (phục vụ mục đích kiểm tra tính toàn vẹn dữ liệu) — đặc tính này lại vô tình giúp kẻ tấn công dò mật khẩu bằng brute-force nhanh hơn. Mật khẩu cần thuật toán băm **chậm một cách có chủ đích**.

```
// NestJS - Password Hashing với bcrypt

import * as bcrypt from 'bcrypt';

async register(email: string, password: string) {

    const hashedPassword = await bcrypt.hash(password, 10); // 10 = độ phức tạp (salt rounds)

    return this.userRepository.save({ email, password: hashedPassword });
}

async validatePassword(plainPassword: string, hashedPassword: string) {

    return bcrypt.compare(plainPassword, hashedPassword);
}
```

6.7 CORS (Cross-Origin Resource Sharing)

Khái niệm: CORS là cơ chế bảo mật do trình duyệt thực thi nhằm kiểm soát việc một trang web ở domain này có được phép gọi API đến server ở domain khác hay không. Mặc định, trình duyệt áp dụng chính sách **Same-Origin Policy** — chặn các request "khác nguồn" (cross-origin, khác domain/port/giao thức) nếu server không tường minh cho phép.

Preflight Request: Với các request "phức tạp" (ví dụ dùng method PUT/DELETE, hoặc có custom header), trình duyệt sẽ tự động gửi trước một request OPTIONS (gọi là preflight) để hỏi server xem request thật sự sắp tới có được phép hay không, trước khi thực sự gửi request chính. Server cần phản hồi đúng các header CORS (Access-Control-Allow-Origin, Access-Control-Allow-Methods...) thì trình duyệt mới cho phép request thật đi tiếp.

Lưu ý quan trọng: CORS chỉ là cơ chế bảo vệ ở phía **trình duyệt**, không bảo vệ được các request gửi trực tiếp từ công cụ như Postman hoặc từ server khác (server-to-server) — vì các công cụ đó không thực thi chính sách CORS. Do đó, CORS không nên được xem là biện pháp bảo mật chính, mà chỉ là lớp bảo vệ bổ sung cho các API được gọi từ trình duyệt.

typescript

```
// NestJS - cấu hình CORS

app.enableCors({

    origin: ['https://myapp.com'], // chỉ cho phép domain này gọi API
```

```
credentials: true,  
});
```

6.8. CSRF (Cross-Site Request Forgery)

Khái niệm: CSRF là hình thức tấn công khiến người dùng đã đăng nhập vô tình thực hiện một hành động ngoài ý muốn trên hệ thống mà họ không hề hay biết, thông qua một trang web độc hại khác. Vì trình duyệt tự động gửi kèm cookie phiên đăng nhập trong mọi request đến domain tương ứng (kể cả khi request đó được kích hoạt từ một trang web khác), kẻ tấn công có thể lợi dụng điều này để giả mạo request (ví dụ: tự động submit một form chuyển tiền ẩn) mà không cần biết mật khẩu của nạn nhân.

Điều kiện để CSRF xảy ra: Hệ thống dùng cookie để duy trì đăng nhập, và không có cơ chế xác minh nguồn gốc của request. Đây là lý do các hệ thống dùng JWT lưu trong Authorization header (không phải cookie) thường ít bị ảnh hưởng bởi CSRF hơn, vì trình duyệt không tự động đính kèm header đó khi request được gửi từ trang khác.

Các cách phòng chống phổ biến:

- **CSRF Token:** Server sinh ra một mã ngẫu nhiên, nhúng vào form hoặc trả về cho client lưu tạm; mọi request thay đổi dữ liệu (POST/PUT/DELETE) phải gửi kèm đúng mã này thì server mới xử lý. Vì trang web độc hại không thể biết được mã này, request giả mạo sẽ bị từ chối.
- **SameSite Cookie:** Thiết lập thuộc tính **SameSite=Strict** hoặc **SameSite=Lax** cho cookie, hạn chế trình duyệt gửi kèm cookie đó trong các request bắt nguồn từ domain khác.
- **Double Submit Cookie:** Một biến thể của CSRF Token, trong đó mã token vừa được lưu trong cookie, vừa phải được gửi kèm tường minh trong request (header hoặc body); server chỉ chấp nhận nếu hai giá trị khớp nhau.

6.9. XSS (Cross-Site Scripting)

Khái niệm: XSS là hình thức tấn công trong đó kẻ tấn công chèn được đoạn mã JavaScript độc hại vào trang web, đoạn mã này sẽ thực thi trên trình duyệt của những người dùng khác truy cập trang đó — có thể dùng để đánh cắp cookie, token lưu trong **localStorage**, hoặc thực hiện hành động giả mạo người dùng.

Ba dạng XSS phổ biến:

Dạng	Mô tả
------	-------

Stored XSS (lưu trữ)	Mã độc được lưu trữ vào cơ sở dữ liệu (ví dụ trong nội dung bình luận), và tự động thực thi mỗi khi có người dùng khác xem lại nội dung đó. Đây là dạng nguy hiểm nhất vì ảnh hưởng đến mọi người xem, không chỉ nạn nhân ban đầu.
Reflected XSS (phản chiếu)	Mã độc nằm ngay trong đường dẫn (URL) hoặc tham số request, được server "phản chiếu" lại nguyên văn trong phản hồi mà không qua xử lý, thường yêu cầu nạn nhân bấm vào một đường dẫn độc hại.
DOM-based XSS	Mã độc không đi qua server, mà được thực thi hoàn toàn ở phía trình duyệt do đoạn JavaScript ở client xử lý dữ liệu đầu vào (ví dụ từ URL) một cách không an toàn.

Cách phòng chống phổ biến:

- Luôn escape (mã hóa ký tự đặc biệt như <, >) dữ liệu do người dùng nhập trước khi hiển thị ra giao diện, để trình duyệt hiểu đó là văn bản thuần túy chứ không phải mã HTML/JavaScript cần thực thi.
- Không tự ý render HTML thô từ dữ liệu người dùng nhập (tránh dùng dangerouslySetInnerHTML trong React nếu không kiểm soát kỹ nội dung, hoặc cần dùng thư viện làm sạch HTML như DOMPurify).
- Thiết lập header **Content-Security-Policy (CSP)** để chỉ định rõ nguồn script/style/hình ảnh nào được phép tải và thực thi trên trang, giúp vô hiệu hóa phần lớn script độc hại ngay cả khi lỡ bị chèn vào trang.

6.10. SQL Injection

Khái niệm: SQL Injection là hình thức tấn công trong đó kẻ tấn công chèn mã SQL độc hại thông qua dữ liệu đầu vào (ví dụ ô tìm kiếm, form đăng nhập), khiến câu truy vấn SQL bị thực thi sai mục đích ban đầu — có thể dẫn đến lộ toàn bộ dữ liệu, chỉnh sửa/xóa dữ liệu trái phép, hoặc trong trường hợp nghiêm trọng là chiếm quyền điều khiển toàn bộ máy chủ cơ sở dữ liệu.

Một số biến thể thường gặp:

- **Classic/In-band SQL Injection:** Kẻ tấn công thấy ngay kết quả khai thác trong phản hồi trả về (ví dụ trang hiển thị lỗi SQL chi tiết, hoặc dữ liệu bị lộ trực tiếp trên giao diện).
- **Union-based:** Lợi dụng câu lệnh **UNION** để gộp thêm một truy vấn khác vào kết quả trả về, qua đó đọc được dữ liệu từ các bảng khác ngoài ý muốn của lập trình viên.

- **Blind SQL Injection:** Ứng dụng không trả về lỗi hay dữ liệu trực tiếp, nhưng kẻ tấn công vẫn có thể suy luận thông tin gián tiếp thông qua việc quan sát ứng dụng phản hồi khác nhau (đúng/sai, nhanh/chậm) tùy vào câu lệnh SQL độc hại được chèn vào.

typescript

```
// SAI - dễ bị SQL Injection vì nối chuỗi trực tiếp
const query = `SELECT * FROM users WHERE email = '${email}'`;

// Nếu email = "" OR '1'='1' => truy vấn trả về TOÀN BỘ người dùng

// ĐÚNG - sử dụng câu lệnh tham số hóa (parameterized query)
const user = await connection.query('SELECT * FROM users WHERE email = ?',
[email]);

// Hoặc dùng ORM (TypeORM/Prisma) - đã tự động tham số hóa câu lệnh
const user = await userRepository.findOne({ where: { email } });
```

Cách phòng chống: luôn sử dụng câu lệnh tham số hóa (parameterized query) hoặc ORM thay vì nối chuỗi trực tiếp dữ liệu người dùng vào câu lệnh SQL; đồng thời áp dụng nguyên tắc **least privilege** — tài khoản kết nối cơ sở dữ liệu của ứng dụng chỉ nên có đúng quyền hạn cần thiết (ví dụ không cấp quyền **DROP TABLE** cho tài khoản chỉ dùng để đọc/ghi dữ liệu thông thường), nhằm giảm thiểu thiệt hại nếu chẳng may bị khai thác.

Lưu ý: các hệ cơ sở dữ liệu NoSQL (như MongoDB) cũng có nguy cơ tương tự, gọi là **NoSQL Injection**, khi dữ liệu đầu vào không được kiểm soát bị dùng trực tiếp để xây dựng câu truy vấn dạng object.

6.11. Helmet

Khái niệm: Helmet là một thư viện middleware phổ biến trong hệ sinh thái Node.js (dùng được cả cho Express và NestJS), giúp tự động thiết lập một loạt các HTTP header liên quan đến bảo mật chỉ với một dòng cấu hình, giúp giảm thiểu nhiều lỗ hổng phổ biến mà không cần cấu hình thủ công từng header.

Một số header tiêu biểu mà Helmet thiết lập:

Header	Tác dụng
--------	----------

Content-Security-Policy	Giới hạn nguồn tài nguyên (script, style, ảnh...) được phép tải, hỗ trợ phòng chống XSS.
X-Frame-Options	Ngăn trang web bị nhúng vào thẻ <iframe> của một trang khác, phòng chống tấn công Clickjacking (đánh lừa người dùng bấm vào phần tử ẩn).
Strict-Transport-Security	Tương đương thiết lập HSTS đã nêu ở mục 6.8, buộc trình duyệt luôn dùng HTTPS.
X-Content-Type-Options	Ngăn trình duyệt tự đoán (sniff) loại nội dung khác với khai báo, tránh một số kiểu khai thác dựa trên việc hiểu sai định dạng tệp.

typescript

```
// Express.js / NestJS

import helmet from 'helmet';

app.use(helmet());
```

CHƯƠNG 7: COMMON BACKEND FEATURES

Bên cạnh các kỹ thuật xử lý nền tảng, hầu hết hệ thống Backend trong thực tế đều cần triển khai một số tính năng lặp lại ở nhiều dự án khác nhau. Chương này trình bày 5 nhóm tính năng phổ biến nhất: Upload file, Webhook, Phân trang dữ liệu, Xóa mềm và Ghi nhật ký thao tác người dùng.

7.1. File Upload

Là tính năng cho phép người dùng gửi tệp (hình ảnh, video, tài liệu...) từ client lên hệ thống Backend để lưu trữ và xử lý.

7.1.1. Multipart Upload

Là hình thức upload cơ bản, tệp được đóng gói trong request dưới định dạng multipart/form-data và gửi trực tiếp lên server trong một lần duy nhất. Đây là cách phổ biến nhất, phù hợp với các tệp có kích thước nhỏ đến trung bình.

typescript

```
// NestJS - Multipart Upload với Multer

@Post('upload')

@UseInterceptors(FileInterceptor('file'))

uploadFile(@UploadedFile() file: Express.Multer.File) {

    return { filename: file.originalname, size: file.size };

}
```

7.1.2. Chunk Upload

Là kỹ thuật chia một tệp lớn thành nhiều phần nhỏ (chunk), gửi lên server lần lượt và ghép lại ở phía server sau khi nhận đủ. Cách này giúp:

- Tránh lỗi khi upload tệp dung lượng lớn (video, file nén...) do giới hạn thời gian hoặc dung lượng request tối đa.
- Cho phép tiếp tục upload từ phần bị lỗi thay vì phải upload lại từ đầu nếu mất kết nối giữa chừng.

7.1.3. Presigned URL

Là một đường dẫn tạm thời, có thời hạn sử dụng ngắn, do server cấp cho client để client **upload trực tiếp lên dịch vụ lưu trữ đám mây** (ví dụ AWS S3), mà không cần dữ liệu tệp phải đi qua server Backend. Cách tiếp cận này giúp giảm tải đáng kể cho Backend (không

phải xử lý luồng dữ liệu tệp lớn), đồng thời tận dụng được băng thông và hạ tầng tối ưu của dịch vụ lưu trữ đám mây.

typescript

```
// NestJS - Tạo Presigned URL với AWS S3

async getUploadUrl(fileName: string) {

  const command = new PutObjectCommand({

    Bucket: 'my-bucket',

    Key: fileName,

  });

  const url = await getSignedUrl(this.s3Client, command, { expiresIn: 300 }); //
  hết hạn sau 5 phút

  return { uploadUrl: url };

}

// Client sau đó dùng URL này để PUT file trực tiếp lên S3, không qua Backend
```

7.1.4. Cloud Storage

Thay vì lưu tệp trực tiếp trên ổ đĩa của server Backend (vốn có nhiều hạn chế: dễ mất dữ liệu khi server gặp sự cố, khó mở rộng khi chạy nhiều instance), các hệ thống hiện đại thường lưu tệp trên các dịch vụ lưu trữ đám mây chuyên dụng (AWS S3, Google Cloud Storage, Cloudinary...). Backend chỉ cần lưu lại đường dẫn (URL) hoặc định danh của tệp trong cơ sở dữ liệu, còn dữ liệu tệp thực tế được quản lý hoàn toàn bởi dịch vụ đám mây.

7.2. Webhook

Khái niệm: Webhook là cơ chế cho phép một hệ thống **chủ động gửi thông báo** (thường dưới dạng HTTP request) đến một hệ thống khác ngay khi có sự kiện xảy ra, thay vì để hệ thống kia phải liên tục hỏi thăm (polling) xem có gì mới hay không. Ví dụ: cổng thanh toán gửi Webhook đến Backend của cửa hàng ngay khi một giao dịch được thanh toán thành công.

7.2.1. Verify Signature (Xác thực chữ ký)

Vì endpoint nhận Webhook thường là một API công khai, cần có cơ chế xác minh rằng request Webhook thực sự đến từ bên gửi hợp lệ (ví dụ: cổng thanh toán), tránh trường hợp kẻ tấn công tự giả mạo request để đánh lừa hệ thống rằng có một giao dịch đã thanh toán thành công. Cách phổ biến là bên gửi sẽ đính kèm một chữ ký (thường tạo bằng HMAC với khóa bí

mật chung) trong header, Backend nhận Webhook sẽ tính toán lại chữ ký từ nội dung request và so sánh với chữ ký nhận được.

typescript

```
// Express.js - Xác thực chữ ký Webhook

import * as crypto from 'crypto';

app.post('/webhook', (req, res) => {

  const signature = req.headers['x-webhook-signature'];

  const expectedSignature = crypto

    .createHmac('sha256', process.env.WEBHOOK_SECRET)

    .update(JSON.stringify(req.body))

    .digest('hex');

  if (signature !== expectedSignature) {

    return res.status(401).json({ message: 'Chữ ký không hợp lệ' });

  }

  // Xử lý sự kiện Webhook...

  res.status(200).send('OK');

});
```

7.2.2. Idempotency

Đối với Webhook, tính chất này đặc biệt quan trọng vì bên gửi thường có cơ chế tự động gửi lại (retry) nếu không nhận được phản hồi thành công từ Backend, dẫn đến việc cùng một sự kiện có thể được gửi đến nhiều lần. Backend cần lưu lại mã định danh của sự kiện đã xử lý (thường có sẵn trong payload Webhook) để bỏ qua các lần gửi trùng lặp, tránh xử lý nghiệp vụ (ví dụ: cộng tiền, gửi email xác nhận) nhiều lần cho cùng một sự kiện.

7.2.3. Queue

Khi nhận được Webhook, Backend nên xác thực chữ ký rồi đưa sự kiện vào Queue để xử lý bất đồng bộ ở phía sau, thay vì xử lý toàn bộ nghiệp vụ ngay trong request. Cách này giúp phản hồi lại cho bên gửi Webhook (ví dụ cổng thanh toán) nhanh chóng — tránh bị bên gửi coi là timeout và gửi lại request không cần thiết — đồng thời tách biệt được nghiệp vụ xử lý phức tạp ra khỏi luồng tiếp nhận Webhook.

7.3. Websocket

Vấn đề của HTTP truyền thống: Giao thức HTTP hoạt động theo mô hình **request-response**: client phải chủ động gửi request thì server mới trả lời được, server không thể tự ý "gửi tin" cho client khi chưa có yêu cầu. Điều này gây khó khăn cho các tính năng cần cập nhật dữ liệu tức thời, ví dụ: tin nhắn chat mới, thông báo realtime, hoặc bảng dữ liệu cập nhật liên tục (dashboard theo dõi đơn hàng). Nếu chỉ dùng HTTP, client buộc phải liên tục tự gửi request hỏi thăm server có gì mới không (kỹ thuật gọi là **Polling**), vừa tốn tài nguyên vừa có độ trễ.

Khái niệm WebSocket: WebSocket là một giao thức cho phép thiết lập một kết nối **hai chiều, duy trì liên tục (persistent)** giữa client và server. Sau khi kết nối được thiết lập (thông qua một bước "bắt tay" ban đầu dựa trên HTTP, gọi là **handshake**), cả client và server đều có thể chủ động gửi dữ liệu cho nhau bất cứ lúc nào, mà không cần phải mở lại kết nối hay gửi kèm request mới — khác biệt căn bản so với HTTP.

So sánh WebSocket với các cách tiếp cận khác:

Kỹ thuật	Cách hoạt động	Đặc điểm
Polling	Client tự động gửi request lặp lại theo chu kỳ (ví dụ mỗi 5 giây) để hỏi có dữ liệu mới không.	Dễ triển khai nhưng lãng phí tài nguyên, có độ trễ bằng với chu kỳ polling.
Long Polling	Client gửi request, server giữ request đó lại cho đến khi có dữ liệu mới mới trả lời, sau đó client lập tức gửi request tiếp theo.	Giảm độ trễ hơn Polling thường, nhưng vẫn tốn tài nguyên do phải duy trì nhiều request chờ.
WebSocket	Thiết lập một kết nối duy nhất, duy trì liên tục, hai chiều.	Độ trễ thấp nhất, tiết kiệm tài nguyên nhất cho các tương tác realtime tần suất cao, nhưng phức tạp hơn khi triển khai

		và mở rộng hệ thống (xem lưu ý bên dưới).
--	--	---

Một số lưu ý khi triển khai WebSocket ở quy mô thực tế:

- **Trạng thái kết nối (Stateful):** Không giống như HTTP là stateless, WebSocket yêu cầu server phải duy trì trạng thái kết nối đang mở cho từng client. Điều này khiến việc mở rộng hệ thống theo chiều ngang (chạy nhiều instance Backend) trở nên phức tạp hơn — cần có cơ chế đồng bộ giữa các instance (thường dùng Redis Pub/Sub) để đảm bảo tin nhắn gửi từ instance này vẫn đến được client đang kết nối ở instance khác.
- **Xác thực kết nối:** Vì WebSocket không dùng lại cơ chế header như HTTP request thông thường trong suốt vòng đời kết nối, việc xác thực (Authentication — mục 6.1) thường phải thực hiện ngay tại thời điểm thiết lập kết nối ban đầu (ví dụ gửi kèm JWT trong bước handshake).
- **Không phải lúc nào cũng cần WebSocket:** Với các tính năng không đòi hỏi độ trễ quá thấp (ví dụ chỉ cần cập nhật thông báo mỗi vài chục giây), Long Polling hoặc **Server-Sent Events (SSE)** — một kỹ thuật cho phép server gửi dữ liệu một chiều đến client qua kết nối HTTP thông thường — có thể là lựa chọn đơn giản hơn mà vẫn đáp ứng đủ nhu cầu.

typescript

```
// NestJS - WebSocket Gateway cơ bản

import { WebSocketGateway, WebSocketServer, SubscribeMessage, MessageBody } from '@nestjs/websockets';
import { Server } from 'socket.io';

@WebSocketGateway({ cors: true })

export class ChatGateway {

  @WebSocketServer()

  server: Server;

  // Lắng nghe sự kiện 'sendMessage' từ client

  @SubscribeMessage('sendMessage')

  handleMessage(@MessageBody() data: { roomId: string; content: string }) {

    // Gửi (broadcast) tin nhắn đến tất cả client đang kết nối trong phòng chat đó
```

```
this.server.to(data.roomId).emit('newMessage', data);

}

handleConnection(client: any) {

    console.log(`Client kết nối: ${client.id}`);

}

}
```

7.4. Pagination

Khái niệm: Pagination là kỹ thuật chia một tập dữ liệu lớn thành nhiều trang nhỏ để trả về cho client, tránh việc phải tải toàn bộ dữ liệu cùng lúc gây chậm và tốn tài nguyên.

Phương pháp	Cách hoạt động	Ưu điểm	Nhược điểm
Offset	Dùng LIMIT và OFFSET để bỏ qua một số dòng nhất định rồi lấy tiếp trang sau.	Dễ triển khai; cho phép nhảy trực tiếp đến bất kỳ số trang nào.	Hiệu năng giảm dần khi OFFSET càng lớn (dữ liệu càng về sau càng chậm); dễ bị trùng/thiếu dữ liệu nếu có bản ghi mới được thêm/xóa giữa các lần load trang.
Cursor	Dùng một "con trỏ" (thường là giá trị của bản ghi cuối cùng ở trang trước) để truy vấn các bản ghi tiếp theo sau con trỏ đó.	Hiệu năng ổn định dù dữ liệu lớn đến đâu; không bị trùng/thiếu dữ liệu khi có thay đổi.	Không thể nhảy trực tiếp đến một trang bất kỳ (chỉ đi tuần tự tiến/lùi).
Keyset	Là một dạng cụ thể của Cursor Pagination, dùng trực tiếp giá trị của một cột có thứ tự rõ ràng (thường là khóa chính hoặc thời gian tạo) làm điều kiện lọc (WHERE id > last_id) thay vì OFFSET.	Hiệu năng rất cao vì tận dụng được Index trên cột lọc; phù hợp với dữ liệu cực lớn.	Cần cột dữ liệu có thứ tự duy nhất, ổn định; phức tạp hơn khi cần sắp xếp theo nhiều tiêu chí.

typescript

```
// Offset Pagination

async getUsersOffset(page: number, limit: number) {

    return this.userRepository.find({

        skip: (page - 1) * limit,

        take: limit,

        order: { id: 'ASC' },

    });

}
```

```
// Keyset Pagination (hiệu quả hơn với dữ liệu lớn)

async getUsersKeyset(lastId: number, limit: number) {

    return this.userRepository

        .createQueryBuilder('user')

        .where('user.id > :lastId', { lastId })

        .orderBy('user.id', 'ASC')

        .take(limit)

        .getMany();

}
```

Gợi ý lựa chọn: Offset phù hợp với giao diện cần hiển thị số trang cụ thể (ví dụ: trang quản trị nội bộ, dữ liệu không quá lớn). Cursor/Keyset phù hợp với các danh sách cuộn vô hạn (infinite scroll) hoặc dữ liệu có quy mô lớn, nơi hiệu năng truy vấn là ưu tiên hàng đầu.

7.5. Soft Delete

Khái niệm: Soft Delete là kỹ thuật "xóa" dữ liệu mà không thực sự loại bỏ bản ghi khỏi cơ sở dữ liệu, thay vào đó chỉ đánh dấu bản ghi là đã bị xóa thông qua một cột thời gian, thường đặt tên là `deletedAt`. Khi truy vấn dữ liệu, hệ thống sẽ tự động lọc bỏ các bản ghi có `deletedAt` khác null.

Soft Delete giúp:

- Có thể khôi phục lại dữ liệu nếu người dùng xóa nhầm.
- Giữ lại lịch sử dữ liệu phục vụ việc thống kê, báo cáo, hoặc tuân thủ yêu cầu lưu trữ dữ liệu.
- Tránh lỗi ràng buộc khóa ngoại khi dữ liệu đã bị tham chiếu ở nhiều bảng khác.

typescript

```
// NestJS + TypeORM - Soft Delete

@Entity()

export class Post {

    @PrimaryGeneratedColumn()

    id: number;

    @DeleteDateColumn() // TypeORM tự quản lý cột deletedAt

    deletedAt: Date;

}

await postRepository.softDelete(id); // UPDATE post SET deletedAt = NOW() WHERE id = ?

await postRepository.find(); // TypeORM tự động thêm điều kiện WHERE deletedAt IS NULL
```

7.6. Audit Log

Khái niệm: Audit Log là bản ghi lưu lại lịch sử các hành động quan trọng xảy ra trong hệ thống, phục vụ mục đích truy vết trách nhiệm, điều tra sự cố, hoặc đáp ứng yêu cầu tuân thủ (compliance) của doanh nghiệp. Khác với Logging thông thường (mục 5.13) vốn phục vụ mục đích gỡ lỗi kỹ thuật, Audit Log tập trung vào khía cạnh nghiệp vụ — trả lời rõ ràng 5 câu hỏi:

- Ai: Người dùng nào đã thực hiện hành động (userId).
- Làm gì: Hành động cụ thể là gì (ví dụ: UPDATE_ORDER, DELETE_USER), thường kèm theo dữ liệu trước và sau khi thay đổi.
- Lúc nào: Thời điểm chính xác hành động xảy ra (timestamp).
- IP: Địa chỉ IP của thiết bị thực hiện hành động, hỗ trợ xác minh nguồn gốc truy cập.
- Device: Thông tin thiết bị/trình duyệt (User-Agent) thực hiện hành động.

typescript

```
// NestJS - Entity và Interceptor ghi Audit Log

@Entity()

export class AuditLog {

  @PrimaryGeneratedColumn()

  id: number;

  @Column() userId: number;      // Ai

  @Column() action: string;      // Làm gì

  @CreateDateColumn() createdAt: Date; // Lúc nào

  @Column() ipAddress: string;    // IP

  @Column() userAgent: string;    // Device
}

@Injectable()

export class AuditLogInterceptor implements NestInterceptor {

  constructor(private auditLogRepository: Repository<AuditLog>) {}

  intercept(context: ExecutionContext, next: CallHandler) {

    const req = context.switchToHttp().getRequest();

    return next.handle().pipe(

      tap(() =>

        this.auditLogRepository.save({

          userId: req.user?.id,

          action: `${req.method} ${req.url}`,

          ipAddress: req.ip,

          userAgent: req.headers['user-agent'],
```

```
        }) ,  
  
        ) ,  
  
        ) ;  
  
    }  
  
}
```